

The Mega Study Guide

One story, two exams — taught from the very beginning

FCS 5DV037 Fundamentals of Computer Science (Datavetenskapens grunder)

PL 5DV086 Programming Languages (Programspråk)

BOTH = on both exams

Contents

Before you start: a letter to you

The story in one page

What the exams look like — so you know what we're aiming at

Your plan

Part I Machines that read — the foundation of both exams

BOTH

1. The language of languages **BOTH**

1.1 The building blocks

1.2 Languages are sets — so we can combine them

1.3 Learning to read the exam's notation

2. The weakest machine: finite automata **BOTH**

2.1 Your first machine

2.2 Getting better at it: remembering the end of the string

2.3 The limit: what sticky notes can't do

2.4 Running two machines at once

2.5 A machine that can guess: NFA

2.6 Guessing is not magic: NFA → DFA

3. Describing patterns: regular expressions **BOTH**

3.1 Reading a regex — the order matters

3.2 Writing a regex — a few moves cover almost everything

3.3 A few equalities worth knowing

3.4 Patterns and machines are the same thing

3.5 Back in the real world **PL**

4. Making machines small: minimization **FCS**

4.1 The marking algorithm

4.2 Worked example

5. Staying regular: closure properties **FCS**

5.1 The four operations the exam asks about

5.2 True/false questions: think of extreme examples

5.3 Questions a machine can always answer

5.4 The ☆ question: an infinite union

6. Proving a language is NOT regular: the pumping lemma **FCS**

6.1 Where the property comes from

6.2 Using it: think of it as a game

7. Grammars: describing things that nest **BOTH**

7.1 A grammar is a set of rewrite rules

7.2 Derivations and trees

7.3 How to design a grammar — think from the outside in

7.4 The real exam questions, solved slowly

7.5 Ambiguity — when one string has two meanings

8. Precedence and associativity **PL**

8.1 Common rules

8.2 How a grammar encodes them

9. Giving the machine a stack: pushdown automata **FCS**

9.1 The machine for $a^n b^n$

9.2 Why the N matters here

9.3 Grammars and stack machines are the same power

10. Chomsky normal form and the CYK algorithm **BOTH**

Part A — Chomsky normal form

Part B — The CYK algorithm

11. Pumping lemma for CFLs, and closure **FCS**

11.1 Closure properties of CFLs

Part II The ultimate machine — and what even it cannot do

FCS

12. Turing machines **FCS**

12.1 How it works

12.2 Watching one work: $a^n b^n$

12.3 Machines that compute answers — and plugging them together

12.4 Upgrades that don't upgrade

12.5 The universal machine

12.6 The whole hierarchy on one page (exam, 8 p)

13. What no machine can do **FCS**

13.1 Two kinds of "solving"

13.2 There are more problems than programs

13.3 The halting problem

13.4 Reductions: spreading the impossibility

14. Fast and slow: P, NP and NP-completeness **FCS**

14.1 Definitions

14.2 SAT → 3SAT → CLIQUE, briefly

14.3 A ☆ exam question: ◇-EQUATION is NP-complete

14.4 A ☆ exam question: NP is closed under $\cup$ and $\cap$

Part III Haskell from zero **PL** — 20 points on every exam

15. Haskell basics: types, functions, lists **PL**

15.1 Using GHCi

15.2 Types

15.3 Defining functions

15.4 Lists: the one data structure you must master

16. Recursion and accumulators **PL**

16.1 Pattern 1 — plain recursion

16.2 Pattern 2 — accumulators and tail recursion

17. Higher-order functions: map, filter, fold, lambda **PL**

17.1 Lambdas, currying, sections, composition

17.2 map and filter — write them yourself

17.3 Folds

17.4 Pipelines — how Haskell programmers really write

18. Data types, type classes, laziness + problem bank **PL**

18.1 Algebraic data types

18.2 Recursive data types and type constructors

18.3 Type classes (ad hoc polymorphism)

18.4 An ADT in Haskell (information hiding via modules)

18.5 Laziness

18.6 Problem bank (verified answers)

Part IV How a program comes alive — ideas behind every language **PL**

19. Names, scope and types **PL**

19.1 Four ways to think about programs

19.2 What a variable really is

19.3 Two meanings of “=” — the functional twist

19.4 Scope: which x is it?

19.5 Types: what may go in the box

20. Control flow and subprograms **PL**

20.1 The minimum you need to compute anything

20.2 Subprograms: abstraction for actions

20.3 Parameter passing: by value, by reference and the others

21. Abstraction, ADTs and object-orientation **PL**

21.1 The two definitions, precisely

21.2 The same stack, showing both ideas (compiles and runs)

21.3 The neighbours — define these precisely too

21.4 Describing a type by what it does, not how

21.5 Object-oriented programming: the big design questions

22. Interpreters: lexer, parser, executor **PL**

22.1 Lexer (scanner)

22.2 Parser

22.3 Between parser and executor: static (semantic) analysis

22.4 Executor (the interpreting part)

22.5 See it: a complete tiny interpreter (Haskell, runs)

22.6 Precedence and associativity (asked on both 2023 exams, 6 p)

23. Doing several things at once — and when things go wrong **PL**

23.1 Concurrency

23.2 Exceptions: when things go wrong

23.3 Events

24. Two other ways to think: functional and logic languages **PL**

24.1 The functional family

24.2 Logic programming: say what, not how

Part V Real exams solved, practice exams, cheat sheets

25. The FCS exam (5DV037), every question answered **FCS**

25.1 Practice exam (new questions)

26. The PL exams (5DV086), every question answered **PL**

26.1 Tentamen 2023-03-16

26.2 Tentamen 2023-05-05

26.3 Your last exam (what you told me)

26.4 Practice exam (60 p, 36 to pass) — 4 hours, no aids

Appendix A. Cheat sheets

Appendix B. Swedish–English glossary (PL exam is in Swedish)

Before you start: a letter to you

I know how this feels. You’ve sat this exam before, you’ve read the slides, you’ve maybe even understood them while reading — and then in the exam room the page stayed half-empty. That isn’t because you can’t do this. It’s because nobody showed you how the pieces *fit together*. Facts you don’t understand slide off the brain in a week. Ideas you understand stay.

So this guide doesn’t hand you facts. It tells you **one story**, from the first page to the last, and every topic on both exams is a chapter of that story. If you follow the story, you won’t need to memorise most of this — you’ll be able to *rebuild* it in the exam room, because it will make sense.

Please read it in order. Every chapter leans on the one before. And please, **do the little “your turn” exercises** with a pen. You don’t learn to swim by reading about water.

You can pass these. Let’s go.

The story in one page

Here is a question that sounds silly but is the heart of computer science:

What can a machine do if it can only remember so much?

We’ll build machines, starting from the weakest imaginable and giving them a little more power each time — and at every step we’ll ask: *what can it do now that it couldn’t before, and what still defeats it?*

The machine gets...	So it can...	But it still can’t...	Where you meet it
a few sticky notes (finite memory)	recognise patterns: “ends in <i>ab</i> ”, valid numbers, identifiers	count without limit (match brackets)	Ch. 2–6: automata, regular expressions — and the lexer in every interpreter
+ a stack of plates	match things that nest: brackets, <code>if / else</code> , blocks	match three things at once	Ch. 7–11: grammars, CNF, CYK — and the parser in every interpreter
+ an unlimited notebook it can erase and reread	compute <i>anything</i> a real computer can	answer some questions about itself (“will this program stop?”)	Ch. 12–13: Turing machines, undecidability
+ a stopwatch	ask not “can it?” but “can it <i>fast</i> ?”	(we don’t know!) — P vs NP	Ch. 14: complexity

And now the twist that ties your two courses together: **a programming language is built out of exactly these machines**. When you press “run”, your code — which is just a long string of characters — goes through:

1. a **lexer**, which is a finite automaton (Ch. 2), chopping text into words;
2. a **parser**, which is a grammar with a stack (Ch. 7–10), finding the structure;
3. an **interpreter**, which walks that structure and makes it happen (Ch. 22) — keeping track of names, scopes, function calls and parameters (Ch. 19–20).

Then we zoom out: how do humans keep huge programs understandable? With **abstraction** — hiding details behind clean walls (Ch. 21). And finally we meet a completely different way to think about

programs: **functional programming in Haskell**, where there are no variables that change and no loops, only definitions (Ch. 15–18) — plus a glimpse of logic programming and concurrency (Ch. 23–24).

HOW TO READ THE BOXES

Intuition — the idea in plain words; read these twice. **Definition** — the precise version you write on the exam. **Recipe** — a step-by-step method you can follow under stress. **Trap** — where students lose points. **Exam** — what the examiner wants to see, often a full model answer. Grey dashed boxes are **your turn** — do them before reading the answer.

What the exams look like — so you know what we’re aiming at

PL Programspråk 5DV086 — 60 points, 36 to pass

Every past exam you have follows the same shape, and it’s a friendly one:

- **~20 points on grammars:** build a context-free grammar, or convert one to Chomsky normal form, or run CYK. (Ch. 7, 10)
- **~20 points of “explain”:** precedence & associativity, the parts of an interpreter, encapsulation & information hiding, pass-by-value vs reference, variables in functional languages. (Ch. 8, 19–22)
- **~20 points of Haskell:** write `myRev`, `compress` / `decompress`, a fold, a list function. (Ch. 15–18)

Some questions appeared **word for word** on two exams in a row. That’s good news: this exam rewards preparation.

FCS Datavetenskapens grunder 5DV037 — 50 points, 25 to pass

Ten questions in four hours: true/false, closure proofs, a table of language classes, a pumping-lemma proof, CNF + CYK, building a Turing machine, and three hard ☆ questions worth only one point each (do them last). You get a sheet with the pumping lemmas and the formal definitions — so you need to *use* them, not memorise them.

Your plan

1. **Part I (Ch. 1–11)** first, whichever exam comes first — both exams stand on it.
2. Then the part for your first exam: **Part II** for FCS, **Parts III–IV** for PL.
3. **Part V:** the real exams, timed, then the practice exams.
4. **Last of all**, the day before: the separate **Priority Pack** — a refresh of exactly what cost you points last time.

Part I

Machines that read — the foundation of both exams

BOTH

1. The language of languages BOTH

Before we build any machine, we need to agree on what a machine is *for*. Here is the simplest possible job: someone hands you a string of symbols, and you must answer **yes** or **no**.

- Is `x1 = 3.14;` a correctly formed Java statement? Yes/no.
- Is `((a+b)*c` correctly bracketed? Yes/no.
- Is `abba` a palindrome? Yes/no.

Every one of these questions is secretly the same question: **is this string in the set of “good” strings?** That set is what we call a **language**. That’s the whole idea. Not English, not Java — just *a set of strings*. Everything in both courses is about describing such sets and building machines that decide membership in them.

1.1 The building blocks

An **alphabet** Σ is the set of symbols we’re allowed to use — like the keys on a keyboard. We’ll usually use $\Sigma = \{a, b\}$ because two symbols are enough to show every interesting idea, and small examples are easier to think about.

A **string** is a finite sequence of symbols: `abba`, `b`, `aaaa`. And there is one very special string: the **empty string**, with no symbols at all. It’s written λ (lambda) in the FCS course and ε (epsilon) on the PL exam. Same thing, two names.

WHY DOES THE EMPTY STRING MATTER SO MUCH?

Because it’s the easiest thing to forget. Is the empty file a valid program? Is “no brackets at all” balanced? Many exam answers are wrong only because they forgot to decide whether λ belongs. Think of λ as the number 0 of strings: gluing it onto anything changes nothing, $\lambda w = w$.

A few pieces of notation you’ll see constantly — each one is simple:

Notation	Means	Example
$ w $	length of w	$ abba = 4$, $ \lambda = 0$
$ w _a$	how many a ’s are in w	$ abba _a = 2$
a^n	n copies of a	$a^3 = aaa$, $a^0 = \lambda$
uv	u followed by v (concatenation)	$u = ab, v = ba: uv = abba$
w^R	w reversed	$(abc)^R = cba$
prefix	a beginning piece	prefixes of abc : λ, a, ab, abc

1.2 Languages are sets — so we can combine them

Σ^* (“sigma star”) is the set of **all** strings over the alphabet, including λ : $\{\lambda, a, b, aa, ab, ba, bb, aaa, \dots\}$. A language is just some subset of it — the strings we call “good”.

Since languages are sets, you can take unions, intersections and complements, exactly as in any maths course. Two more operations are special to languages, and you should picture them rather than memorise them:

- **Concatenation** $L_1 L_2$: pick one string from L_1 , one from L_2 , glue them. Every combination.
 $\{a, ab\}\{b\} = \{ab, abb\}$.
- **Star** L^* : glue together *any number* of strings from L — including zero of them, which gives λ .
 $\{ab\}^* = \{\lambda, ab, abab, ababab, \dots\}$. Think of it as “repeat as often as you like”.

THREE KINDS OF “NOTHING”

$\emptyset$ is a language with **no strings** (an empty bag). $\{\lambda\}$ is a language with **one** string, the empty one (a bag containing an empty envelope). λ is a **string** (the empty envelope itself). They are different, and examiners love checking whether you know it.

1.3 Learning to read the exam’s notation

Exam questions describe languages in **set-builder notation**: $\{w \mid \text{condition}\}$ means “all strings w for which the condition holds.” The single most useful habit I can give you:

THE 30-SECOND HABIT THAT SAVES POINTS

Before you build anything for a language, write down **three strings that are in it** and **three that are not** — starting with the shortest ones, and always deciding about λ . If you can’t do this, you don’t understand the question yet, and nothing you build will be right.

Let’s practise on languages that really appear on the exams. Cover the right-hand columns and try first.

Language	In words	In it	Not in it
$\{a^n b^n \mid n \geq 0\}$	a block of a’s, then the same number of b’s	$\lambda, ab, aabb$	$aab, ba, abab$
$\{a^n b^m \mid n \neq 2m\}$	a’s then b’s, and #a is not twice #b	$a, b, ab, aaab$	$\lambda, aab, aaaabb$
$\{w \mid w _a = w _b\}$	equally many a’s and b’s, in any order	$\lambda, ba, abba$	a, aab
$\{ww^R\}$	a string followed by its mirror image	$\lambda, aa, abba$	aba, ab

Look closely at the first and third rows. $a^n b^n$ *forces the order* — all a’s first. $|w|_a = |w|_b$ doesn’t care about order at all: ba is fine. Confusing those two is one of the most common reasons a grammar answer gets zero.

YOUR TURN

1. Is λ in $\{a^n b^m \mid n \neq 2m\}$?
2. Write all strings in $\{a, b\}^*$ of length 2.
3. What is $\{a, b\}\{\lambda, a\}$?
4. Give three members of $\{w \mid w \text{ has an odd number of } b\text{'s}\}$.

ANSWERS

1. No — $\lambda = a^0 b^0$ and $0 = 2 \cdot 0$, so the condition fails.
2. aa, ab, ba, bb .
3. $\{a, aa, b, ba\}$.
4. For example b, ab, bbb — any string where the number of b’s is odd.

Where we’re going next. We now know what a language is. The next question is the one that drives the whole course: *what is the simplest machine that can decide whether a string belongs to a language?* Let’s

build it.

2. The weakest machine: finite automata BOTH

Imagine you work at a door. People hand you a paper strip with letters on it, and you read the letters **one at a time, left to right, and you can never go back**. At the end you must say “let them in” or “no”. The catch: you have no paper to write on. All you have is a small number of **sticky notes**, and at any moment exactly one of them is on your forehead, reminding you of *where you are*.

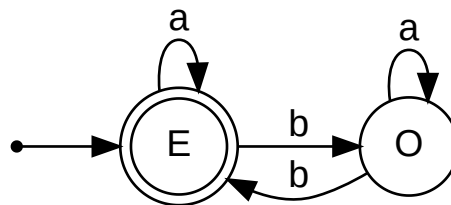
That is a finite automaton. The sticky notes are its **states**. And the entire skill of designing one comes down to a single question — the most important question in this chapter:

“What is the smallest thing I need to remember about what I’ve read so far?”

2.1 Your first machine

Job: let a string in if it has an **even number of b’s**.

What do you need to remember? Not the whole string. Not even how many b’s — just whether the count so far is *even* or *odd*. Two sticky notes: **E** and **O**. You start at E (zero b’s is even). An *a* changes nothing. A *b* flips you.



The double circle marks the 'let them in' states. The arrow from nowhere marks where you start.

Let’s walk *abba* through it: start $E \rightarrow a \rightarrow E \rightarrow b \rightarrow O \rightarrow b \rightarrow E \rightarrow a \rightarrow E$. We end on E, a double circle: accepted. Now try *ab* yourself — you should end on O and reject.

That drawing is the machine. On the exam you describe it formally like this:

DEFINITION — DFA (DETERMINISTIC FINITE AUTOMATON)

$M = (Q, \Sigma, \delta, q_0, F)$: Q the finite set of states (sticky notes), Σ the alphabet, $\delta : Q \times \Sigma \rightarrow Q$ the transition function (“in this state, reading this letter, go there”), q_0 the start state, $F \subseteq Q$ the accepting states. The language of M is the set of strings that lead from q_0 to a state in F .

Deterministic means there is **exactly one** arrow for every state and every letter — never a choice, never a missing arrow.

“Never a missing arrow” — what if some letter should just kill the string? Then you add a **trap state**: a non-accepting state that loops to itself on everything. Once you fall in, you never get out. (Many drawings leave the trap state out and say “missing arrows go to the trap” — that’s fine if you say it.)

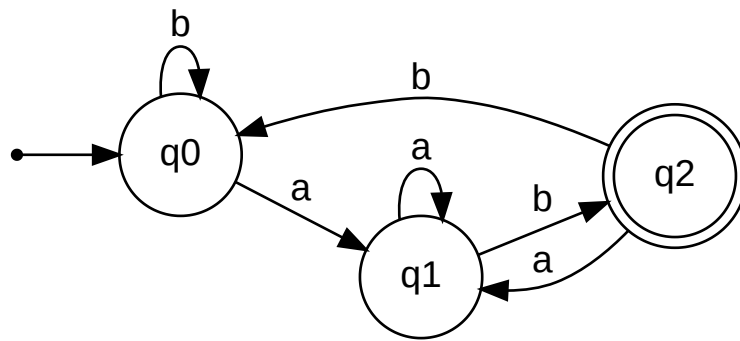
2.2 Getting better at it: remembering the end of the string

Job: accept strings that **end with *ab***.

What do I need to remember? How much of “*ab*” I’ve *just* seen:

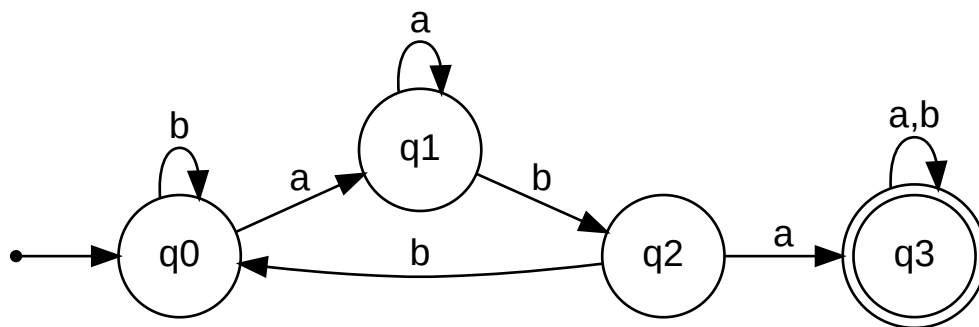
- q_0 : nothing useful yet (the string doesn’t end in *a* or *ab*),

- q_1 : I just saw an a — half-way there,
- q_2 : I just saw ab — if the string stopped now, accept.



The interesting arrows are the ones that “fall back”. Suppose we’re in q_2 (just saw ab) and read an a . Is all hope lost? No! The string now ends in a — we’re half-way again, so we go to q_1 , not q_0 . And from q_1 , another a keeps us in q_1 : aa still ends in a . Getting those fall-back arrows right is where most DFA points are won or lost. Always ask: “*after this letter, what does the string end with?*”

One more: “contains aba somewhere”



Same idea, with one new twist: once you’ve seen aba , the answer is yes **forever**, whatever comes after. So q_3 just loops. That’s a “success trap”.

RECIPE: DESIGNING A DFA

1. Ask “what must I remember?” — a parity, a remainder, the last few letters, a flag “already seen X”.
2. One state per thing-to-remember. Accepting = “if the input stopped now, I’d say yes”.
3. For **every** state and **every** letter, ask “what do I remember now?” and draw that arrow.
4. Is λ in the language? Then the start state must be accepting.
5. Test on 3–4 short strings.

2.3 The limit: what sticky notes can’t do

Try to build a DFA for $\{a^n b^n\}$ — some a ’s, then exactly as many b ’s. You’d need to remember *how many* a ’s you saw: 1, 2, 3, ... 1000, ... — one sticky note per number, and there’s no end to the numbers. But a DFA has a *fixed, finite* number of notes. So it can’t be done. Hold on to that intuition: **finite memory can’t count without bound**. In Chapter 6 we’ll turn it into a proof, and in Chapter 7 we’ll give the machine more memory to fix it.

2.4 Running two machines at once

Suppose you want strings with an even number of b's **and** ending in *ab*. You already have both machines. Just run them **side by side**, one finger on each: your state is now a *pair* (where machine 1 is, where machine 2 is). Accept when **both** accept. That's the **product construction**:

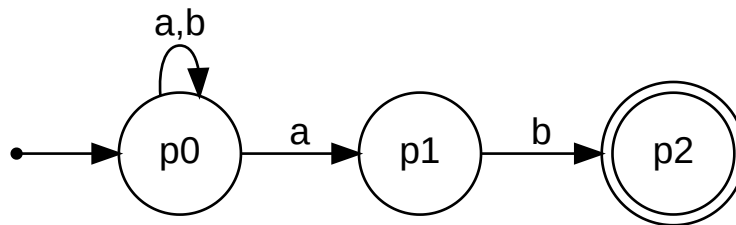
PRODUCT CONSTRUCTION

States $Q_1 \times Q_2$; start (q_1, q_2) ; $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$. For **and** ($\cap$): accept when both parts accept. For **or** ($\cup$): accept when at least one does.

This little trick is the proof that regular languages are closed under union and intersection — an exam question we'll answer properly in Chapter 5.

2.5 A machine that can guess: NFA

Designing “ends with *ab*” took some thinking about fall-back arrows. What if the machine could simply **guess**? “I'll keep reading... and *now* I bet the final *ab* starts.” If the guess was right, it accepts:



From p_0 on *a* there are **two** arrows — a choice. From p_1 on *a* there is **no** arrow — that guess just dies. This is a **nondeterministic** automaton. The rule: it **accepts if at least one way of guessing reaches an accepting state**. Picture it cloning itself at every choice; if any clone ends happy, the answer is yes.

DEFINITION — NFA

Like a DFA, but $\delta : Q \times (\Sigma \cup \{\lambda\}) \rightarrow 2^Q$: each state and letter gives a **set** of possible next states (maybe empty), and λ -arrows can be taken without reading anything. w is accepted if **some** path spelling w ends in an accepting state.

2.6 Guessing is not magic: NFA $\rightarrow$ DFA

Here is a surprising and beautiful fact: guessing gives **no extra power**. Any NFA can be turned into a DFA that accepts the same strings. The trick: instead of guessing, **keep track of every place the NFA could be right now**. Your sticky note no longer says “ p_0 ” — it says “ $\{p_0, p_1\}$ ”. There are only finitely many such sets, so this is still a DFA.

RECIPE: SUBSET CONSTRUCTION

1. Start state = $\{q_0\}$ plus everything reachable from it by λ -arrows (the **λ -closure**).
2. For a set S and a letter: collect everywhere any state in S can go on that letter, then add the λ -closure.
3. Repeat for every new set that appears. The empty set is a trap state.
4. A set is accepting if it **contains** an accepting NFA state.

Let's do it for the guessing machine above. Start at $\{p_0\}$.

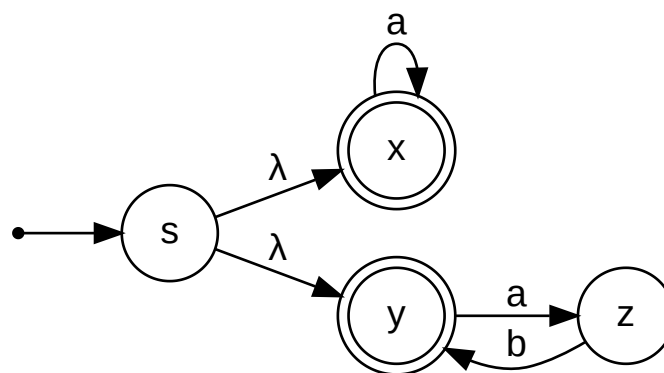
DFA state	on a	on b	accepting?
$\rightarrow \{p_0\}$	$\{p_0, p_1\}$	$\{p_0\}$	
$\{p_0, p_1\}$	$\{p_0, p_1\}$	$\{p_0, p_2\}$	
$\{p_0, p_2\}$	$\{p_0, p_1\}$	$\{p_0\}$	yes (contains p_2)

Read the first row slowly: from p_0 , an a could keep us in p_0 or move us to p_1 — so the new set is $\{p_0, p_1\}$. Second row, on b : p_0 stays at p_0 , p_1 moves to $p_2 \rightarrow \{p_0, p_2\}$.

Now look at what we got: three states, and if you rename $\{p_0\} = q_0$, $\{p_0, p_1\} = q_1$, $\{p_0, p_2\} = q_2$, it is **exactly** the DFA we designed by hand in 2.2, fall-back arrows and all. The construction figured out the tricky arrows for us.

With λ -arrows

NFA for “only a’s, or repetitions of ab ” ($a^* \cup (ab)^*$):



DFA state	on a	on b	accepting?
$\rightarrow \{s, x, y\}$	$\{x, z\}$	$\emptyset$	yes
$\{x, z\}$	$\{x\}$	$\{y\}$	yes
$\{x\}$	$\{x\}$	$\emptyset$	yes
$\{y\}$	$\{z\}$	$\emptyset$	yes
$\{z\}$	$\emptyset$	$\{y\}$	
$\emptyset$	$\emptyset$	$\emptyset$	

Notice the start state: not $\{s\}$ but $\{s, x, y\}$, because s can slide along λ -arrows to x and y for free. Forgetting that closure is the number-one mistake in this algorithm.

WHERE STUDENTS LOSE POINTS HERE

- Forgetting the λ -closure (of the start state, and after every step).
- Leaving out $\emptyset$ — it's a real state of the DFA, with arrows to itself.
- Marking a set accepting only if it “is” final — the rule is: it **contains** a final state.

YOUR TURN

1. DFA for strings where every a is immediately followed by a b .
2. DFA for strings of odd length.
3. NFA for “second-to-last letter is a ”, then convert it to a DFA.

ANSWERS

1. Remember: “all fine” (start, accepting), “owe a b ” (just read a), and a trap. fine $-a \rightarrow$ owe, fine $-b \rightarrow$ fine, owe $-b \rightarrow$ fine, owe $-a \rightarrow$ trap.
2. Two states that swap on every letter; the “odd” one accepts.
3. p_0 loops on a, b ; $p_0 -a \rightarrow p_1$; $p_1 -a, b \rightarrow p_2$ (accepting). The DFA has four states $\{p_0\}, \{p_0, p_1\}, \{p_0, p_2\}, \{p_0, p_1, p_2\}$ — they remember the last two letters — and the ones containing p_2 accept.

Where we’re going next. Drawing machines is one way to describe a language. There’s a second way, which you’ve probably used in programming without knowing its theory: writing a *pattern*. That’s a regular expression — and it turns out to describe exactly the same languages.

3. Describing patterns: regular expressions BOTH

A machine *checks* strings. But often you just want to *describe* them: “an identifier is a letter followed by letters or digits.” For that we use a tiny language of patterns with only **three operations** — and it’s worth pausing on how little that is:

- **or** — $a + b$: either an a or a b (the FCS course writes “or” as $+$);
- **then** — ab : an a followed by a b ;
- **repeat** — a^* : zero or more a ’s.

Plus the building blocks: single letters, λ (the empty string) and $\emptyset$ (nothing). That’s the whole language. Everything else is combining these.

DEFINITION

$\emptyset$, λ and each letter a are regular expressions. If r_1, r_2 are, so are $r_1 + r_2$ (union), $r_1 r_2$ (concatenation), r_1^* (star) and (r_1) . Nothing else is.

“+” MEANS TWO DIFFERENT THINGS IN YOUR TWO COURSES

In the FCS course, $a + b$ means “a **or** b”. In programming (Python, Java, grep), “or” is $a|b$ and $a+$ means “**one or more** a’s”. Same symbol, different meaning. On the PL exam, say which notation you use.

3.1 Reading a regex — the order matters

Just like in maths, where $2 + 3 \times 4$ means $2 + (3 \times 4)$, regular expressions have a precedence order: **star binds tightest, then “then”, then “or”**.

You write	It means	Not
ab^*	$a(b^*)$ — $a, ab, abb, \dots$	$(ab)^*$
$a + bc$	$a + (bc)$ — just a or bc	$(a + b)c$

Keep this in the back of your mind — it’s the exact same idea as operator precedence in programming languages (Chapter 8).

Now read some patterns. The trick is to translate them into a sentence:

Regex	Say it in words
$(a + b)^*$	“any string at all”
$(a + b)^* ab$	“anything, then ab ” = ends with ab
$(a + b)^* aba(a + b)^*$	“anything, aba , anything” = contains aba
$a^* ba^*$	exactly one b
$a^*(\lambda + b)a^*$	at most one b (the $\lambda + b$ means “a b , or nothing”)
$((a + b)(a + b))^*$	chunks of two letters = even length
$a^*(ba^*ba^*)^*$	b ’s come in pairs = even number of b ’s
$(b + ab)^*(\lambda + a)$	every a is followed by b , except maybe at the very end = no aa

3.2 Writing a regex — a few moves cover almost everything

THE MOVES

- **contains X:** $(a + b)^* X (a + b)^*$. **starts with X:** $X (a + b)^*$. **ends with X:** $(a + b)^* X$.
- **exactly two b's:** put them in explicitly, with “anything but b” around them: $a^* b a^* b a^*$.
- **counting modulo k:** a chunk containing exactly k of the letter, repeated: even b's = $a^* (b a^* b a^*)^*$.
- **A or B:** build each, join with +.

A real exam statement (FCS, true/false): “the strings over $\{a, b, c\}$ that start with a , or end with c , or have an odd number of b 's form a regular language.” **True** — each of the three parts is easy to write, and “or” is just +:

$$a(a + b + c)^* + (a + b + c)^* c + (a + c)^* b (a + c)^* (b (a + c)^* b (a + c)^*)^*$$

3.3 A few equalities worth knowing

- $\lambda r = r$ (gluing on nothing changes nothing) — but $\emptyset r = \emptyset$ (gluing onto *no* string gives no string).
- $\emptyset^* = \lambda$: repeating nothing zero times still gives the empty string.
- $(r^*)^* = r^*$, $\lambda + r r^* = r^*$.
- $(ab)^* a = a (ba)^*$ — both are $a, aba, ababa, \dots$
- **Trap:** $(a + b)^* \neq a^* + b^*$. The left allows ab ; the right is “only a's or only b's”.

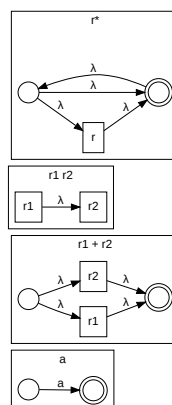
3.4 Patterns and machines are the same thing

Here's the big result of the last two chapters, and it's genuinely remarkable:

KLEENE'S THEOREM

A language can be described by a regular expression **if and only if** it can be recognised by a DFA (or NFA). Such languages are called **regular**.

Why is it true? In one direction, you can build an NFA from a regex piece by piece, the same way you'd evaluate an arithmetic expression from the inside out:



Small NFAs for a letter, for 'or', for 'then' and for 'repeat', glued with λ -arrows

- **or:** a new start state with λ -arrows into both machines — “guess which one”.
- **then:** a λ -arrow from the end of the first machine to the start of the second.

- **repeat:** a way to skip straight to the end (zero times) and a λ -arrow back to go again.

In the other direction, you can take any automaton and remove its states one by one, writing the paths through each removed state as small regexes on the arrows, until one arrow with the whole regex remains. You don't need to be fast at that — you need to know that it works, because that's what makes the theorem true.

3.5 Back in the real world PL

Why should a programmer care? Because the very first thing an interpreter does with your code is chop it into “words” — numbers, names, operators — and each kind of word is described by a regular expression:

Token	Pattern	In course notation
integer	<code>[0-9]+</code>	dd^* with $d = 0 + 1 + \dots + 9$
identifier	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	letter, then (letter + digit)*
optional sign	<code>-?</code>	$\lambda + -$

The part of an interpreter that does this — the **lexer** — is literally a finite automaton built from those patterns. You'll see it at work in Chapter 22.

YOUR TURN

1. Exactly two a 's.
2. Does not end in b .
3. Is $(a + b)^*a(a + b)^*$ equal to $(a + b)^*$?

ANSWERS

1. $b^*ab^*ab^*$.
2. $\lambda + (a + b)^*a$ — the λ is easy to forget!
3. No: the first needs at least one a , so λ and bbb are missing.

Where we're going next. Now that we can build machines, two natural questions: can we make a machine as *small* as possible (Chapter 4)? And which operations keep us inside the world of regular languages (Chapter 5)?

4. Making machines small: minimization FCS

Two people design a DFA for the same job. One has 3 states, the other has 7. Are they really different machines? Often the 7-state one just has some states that are **secretly the same** — sticky notes with different names that mean the same thing. Minimization finds those and merges them.

WHEN ARE TWO STATES “THE SAME”?

When **the future looks identical** from both. If, from state p and from state q , every possible rest-of-the-string leads to the same verdict (accept/reject), then it doesn't matter which of them you're in. They're **indistinguishable**.

If there is *some* string that leads one to accept and the other to reject, that string **distinguishes** them — they must stay separate.

4.1 The marking algorithm

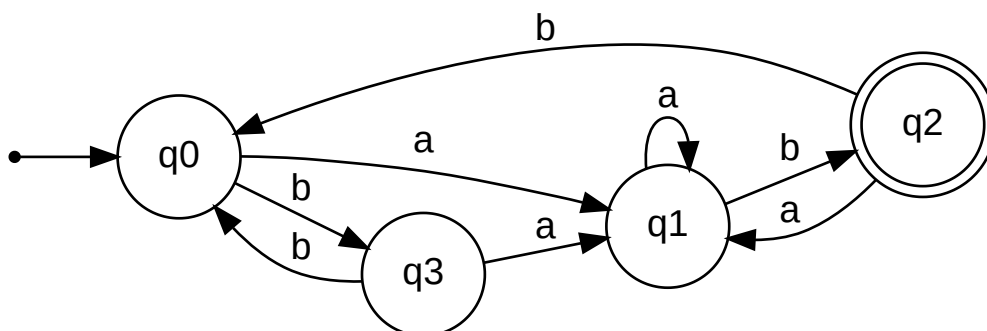
Rather than hunting for strings that are equal, we hunt for **differences**, starting from the obvious one and letting them spread:

RECIPE

1. Delete states you can't reach from the start.
2. Make a table with a cell for every **pair** of states.
3. **Round 0:** mark every pair where one is accepting and the other isn't. (The empty string already tells them apart.)
4. **Next rounds:** for each unmarked pair (p, q) and each letter: if the letter leads them to a pair that is **already marked**, mark (p, q) too — that letter plus the old distinguishing string tells them apart.
5. Stop when a round marks nothing new. Every unmarked pair can be merged.

4.2 Worked example

This machine accepts “ends with ab ”, but has four states where three would do:



Round	What happens
0	q_2 is the only accepting state, so mark (q_0, q_2) , (q_1, q_2) , (q_2, q_3) .
1	(q_0, q_1) : on b they go to (q_3, q_2) — marked! So mark (q_0, q_1) . (q_1, q_3) : on b they go to (q_2, q_0) — marked! Mark it. (q_0, q_3) : on a both go to q_1 ; on b both go to q_0 . Nothing marked. Stays unmarked.
2	Only (q_0, q_3) is left; its answers haven't changed. Nothing new — stop.

Result: q_0 and q_3 are the same state. Merge them and you get exactly the 3-state machine from Chapter 2. Makes sense: both mean “nothing useful seen yet”.

TWO FACTS WORTH KNOWING

The smallest DFA for a regular language is **unique** (apart from state names). So to check if two DFAs accept the same language, minimise both and compare — which means equivalence of regular languages is **decidable**.

TRAPS

Stopping after one round (keep going until nothing changes). Forgetting the trap state if the drawing has missing arrows. Forgetting to remove unreachable states first.

5. Staying regular: closure properties FCS

Suppose you have two regular languages — two things you can recognise with sticky-note machines. You combine them: their union, their intersection, one after the other, the complement. **Is the result still something a sticky-note machine can recognise?**

If the answer is always yes, we say regular languages are **closed** under that operation. It's like asking: "if I add two whole numbers, do I always get a whole number?" (yes — closed). "If I divide them?" (no — $1/2$).

HOW DO YOU PROVE "ALWAYS YES"?

You can't test all languages. Instead you give a **recipe**: "give me machines (or regexes) for L_1 and L_2 , and here is how I build one for the result." If the recipe always works, you've proven closure. Every proof in this chapter is a recipe you already know.

5.1 The four operations the exam asks about

Union and concatenation — use regexes. If r_1 and r_2 describe L_1 and L_2 , then $r_1 + r_2$ describes the union and $r_1 r_2$ describes the concatenation. Done — a regex exists, so the language is regular. (Star works the same way: r_1^* .)

Complement — flip the verdict. Take a DFA for L . Run it exactly as before, but **swap** which states are accepting. Every string that used to be accepted is now rejected and vice versa. That machine recognises $\overline{L}$.

THIS ONLY WORKS WITH A COMPLETE DFA

With an NFA, swapping fails: a string might have one path ending in an accepting state and another ending in a non-accepting state, so it would be accepted both before and after swapping. And the DFA must have all its arrows (include the trap state!), or strings that fell off a missing arrow would be rejected both times.

Intersection — run both machines side by side. The product construction from Chapter 2: states are pairs, accept when **both** accept. (Or a slicker argument: $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$ by De Morgan's law — and we already have complement and union.)

EXAM: "ARGUE THAT REGULAR LANGUAGES ARE CLOSED UNDER COMPLEMENT, INTERSECTION, CONCATENATION AND UNION" (5 P)

Let L_1, L_2 be regular, described by regular expressions r_1, r_2 and recognised by complete DFAs $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.

Union / concatenation: $r_1 + r_2$ and $r_1 r_2$ are regular expressions for $L_1 \cup L_2$ and $L_1 L_2$, so these are regular.

Complement: $(Q_1, \Sigma, \delta_1, q_1, Q_1 \setminus F_1)$ follows the same path as M_1 on every input but accepts exactly when M_1 rejects, so it recognises $\overline{L_1}$. (M_1 must be complete — add a trap state if needed.)

Intersection: the product DFA $(Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F_1 \times F_2)$ with $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$ simulates both machines at once and accepts exactly when both accept, i.e. it recognises $L_1 \cap L_2$.

Also closed: difference ($L_1 \cap \overline{L_2}$) and reversal (turn all arrows around).

5.2 True/false questions: think of extreme examples

The FCS exam has true/false questions where a wrong answer **costs** half a point. The trick for any statement with “necessarily” or “always”: to show it’s false, you need just **one** counterexample — and the best counterexamples are the extreme languages: $\emptyset$ (nothing) and Σ^* (everything).

Statement	Answer and the example that settles it
L context-free but not regular, L' regular $\Rightarrow L \cup L'$ is not regular	False: take $L' = \Sigma^*$. The union is everything — regular.
...same, $\Rightarrow L \cap L'$ is not regular	False: take $L' = \emptyset$. The intersection is empty — regular.
Every finite language is regular	True: list its strings joined by +.
Every subset of a regular language is regular	False: $\{a^n b^n\} \subseteq \Sigma^*$.

5.3 Questions a machine can always answer

For regular languages, these questions all have algorithms (they are **decidable**):

- **Is w in L ?** Run the DFA.
- **Is L empty?** Can you reach any accepting state from the start? (Follow the arrows.)
- **Is L infinite?** Is there a **loop** on some path from the start to an accepting state? A loop means you can go round it as often as you like and still accept — infinitely many strings.
- **Are L_1 and L_2 equal?** Minimise both DFAs and compare (Chapter 4).

5.4 The ☆ question: an infinite union

We know: union of two regular languages is regular. So union of three is too, and of a hundred. What about an **infinite** union? It’s tempting to say “yes, obviously” — and that’s exactly the trap.

EXAM ☆: “EACH L_N IS REGULAR. IS $\bigcup_{N \in \mathbb{N}} L_N$ NECESSARILY REGULAR?”

No. Let $L_n = \{a^n b^n\}$ — a language with a single string, so it’s finite and therefore regular. But the union of all of them is $\{a^n b^n \mid n \geq 0\}$, which is not regular (Chapter 6). Closure under union only covers **finitely** many languages.

Where we’re going next. We keep saying “ $a^n b^n$ is not regular”. Time to *prove* it.

6. Proving a language is NOT regular: the pumping lemma FCS

It's easy to show a language *is* regular: build the machine. But how do you show that **no** machine exists — not one you failed to find, but one that *cannot* exist? You need a property every regular language has, and then show your language doesn't have it.

6.1 Where the property comes from

THE PIGEONHOLE IDEA

Take a DFA with, say, 5 states. Feed it a string of length 5 or more. Reading 5 letters it passes through 6 states — but there are only 5. So **some state repeats**. (Six pigeons, five holes: two pigeons share a hole.)

The letters read between the two visits to that state form a **loop** in the machine. Now here's the key: the machine can't tell how many times it went around that loop. Go around zero times, twice, a hundred times — it ends up in the same place at the end, so it gives the same verdict.

So in every long enough accepted string there's a piece near the start — call it y — that you can **pump**: delete it, or repeat it, and the string stays in the language.

Written formally (you get this on the exam's definition sheet):

PUMPING LEMMA FOR REGULAR LANGUAGES

If L is regular, there is a number n such that every $w \in L$ with $|w| \geq n$ can be split as $w = xyz$ with $|xy| \leq n$, $|y| \geq 1$, and $xy^iz \in L$ for **every** $i \geq 0$.

(n is the number of states; $|xy| \leq n$ because the repeat happens within the first n letters; $|y| \geq 1$ because a loop reads at least one letter.)

6.2 Using it: think of it as a game

To prove L is **not** regular, you play a game against an opponent who claims it is — and you show you can always win:

1. **Opponent**: "It's regular, and my pumping number is n ." (You don't know n — work with the letter.)
2. **You**: pick a string w in L of length at least n . **This is your big move**. Choose it so the first n letters are all the same — then the opponent's y is trapped there.
3. **Opponent**: splits $w = xyz$ any way they like, as long as $|xy| \leq n$, $|y| \geq 1$. You must beat **every** possible split.
4. **You**: pick how many times to pump ($i = 0$ or $i = 2$ usually) so the result falls out of L .
5. You won $\Rightarrow$ contradiction $\Rightarrow L$ is not regular.

The classic: $\{a^k b^k \mid k \geq 0\}$

Assume L is regular with pumping number n . Choose $w = a^n b^n$ — it's in L and has length $2n \geq n$. Whatever split the opponent chooses, $|xy| \leq n$ means x and y sit inside the first n letters, which are

all a 's. So $y = a^j$ for some $j \geq 1$. Pump with $i = 2$: $xy^2z = a^{n+j}b^n$ — more a 's than b 's, not in L . Contradiction, so L is not regular.

Read that proof again and notice how little there is to it. The choice $w = a^n b^n$ did all the work: it forced y to be only a 's.

More, same pattern

- $\{ww^R\}$: choose $w = a^n b b a^n$. y is a 's from the first block; $i = 0$ makes the halves unequal — not a palindrome.
- $\{a^n b^m \mid n > m\}$: choose $w = a^{n+1} b^n$; $i = 0$ removes at least one a , leaving $\#a \leq \#b$.
- $\{a^{k^2}\}$: choose $w = a^{n^2}$; $i = 2$ gives length between n^2 and $n^2 + n < (n+1)^2$ — not a square.
- $\{w \mid |w|_a = |w|_b\}$: a shortcut! If it were regular, so would be its intersection with the regular $a^* b^*$ — but that's $\{a^n b^n\}$. Closure properties can prove non-regularity too.

HOW THESE PROOFS LOSE POINTS

- Picking a concrete number like $w = a^5 b^5$. Your string must use n .
- Choosing the split yourself ("let $y = a$ "). The opponent chooses; you must handle all splits.
- Trying to use the lemma to prove a language **is** regular. It can't — it only rules things out.
- Forgetting the last sentence: "contradiction, so L is not regular."

Where we're going next. So finite memory can't count. How much memory do we need to add to match a 's with b 's — and brackets with brackets? Surprisingly little: a stack. And the way to *describe* such languages is one of the most useful tools in all of computer science: the grammar.

7. Grammars: describing things that nest BOTH

Think about brackets in code: `{ if (x) { while (y) { ... } } }`. Every opening bracket needs its partner, and they nest inside each other. We just proved finite machines can't handle this. So how do programming languages describe their own syntax? With a **grammar**. This chapter is worth more points than any other on the PL exam — so let's take it slowly.

7.1 A grammar is a set of rewrite rules

THE GAME

You start with one symbol, S . You have rules like $S \rightarrow aSb$ (“you may replace S by aSb ”). You keep replacing capital letters using the rules until only small letters are left. Every string you can end up with is in the language.

Let's play with the grammar $S \rightarrow aSb \mid \lambda$ (the bar $|$ means “or”: two rules, $S \rightarrow aSb$ and $S \rightarrow \lambda$):

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaabbb$$

Look at what happened. Each time we used $S \rightarrow aSb$, an a appeared on the left **and** a b on the right, **at the same time**. The a 's and b 's are created in pairs, wrapped around the middle. So the counts are always equal: this grammar generates exactly $\{a^n b^n\}$ — the language no finite machine could handle!

That's the superpower of grammars in one sentence: **a rule can put things on both sides at once**, which is exactly how you match partners that are far apart — a 's with b 's, `(` with `)`, `begin` with `end`.

DEFINITION — CONTEXT-FREE GRAMMAR

$G = (V, T, S, P)$ — written $G = (N, \Sigma, R, S)$ on the PL exam: **variables** (non-terminals, capital letters), **terminals** (the alphabet, small letters), a **start variable** S , and **productions** $A \rightarrow x$ where the left side is a **single variable**. “Context-free” means you may rewrite A whatever is around it.

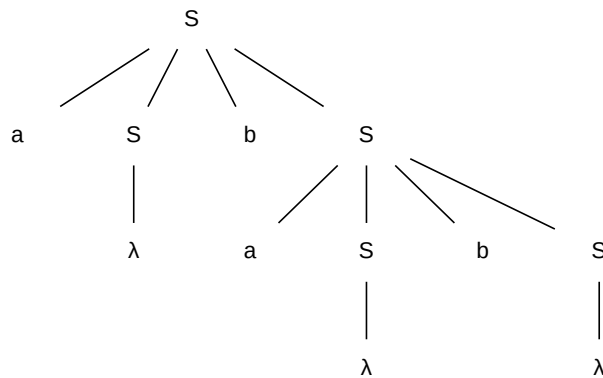
The language $L(G)$ is all terminal strings you can derive from S .

7.2 Derivations and trees

A **derivation** is the sequence of rewrites. When there are several variables to choose from, a **leftmost derivation** always rewrites the leftmost one. Example with $S \rightarrow aSbS \mid \lambda$ and the string $abab$:

$$S \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow abab$$

The same thing drawn as a **parse tree** — each rewrite becomes a node with its children:



Read the leaves left to right and you get the string. **This tree is what a parser builds from your code** — hold on to that picture, it comes back in Chapter 22.

7.3 How to design a grammar — think from the outside in

Designing grammars feels hard only until you learn to see a few shapes. Here is the way of thinking I want you to practise:

THE METHOD

1. Write 3 members and 3 non-members (Chapter 1's habit).
2. Look for things that must **match in number**. If they are on opposite sides, create them together with one rule: $S \rightarrow aSb$.
3. Whatever sits in the middle, or is free, gets its own variable.
4. “or” in the description = separate alternatives $S \rightarrow S_1 \mid S_2$. “followed by” = $S \rightarrow S_1S_2$.
5. Test: shortest member (λ ?), two longer members, one non-member.

Let's grow a family of grammars, each a small change of the last. Watch how the idea evolves:

Step 1 — exactly matched: $a^n b^n$

$S \rightarrow aSb \mid \lambda$. One a, one b, at the same time.

Step 2 — a different ratio: $a^n b^{2n}$

Every a needs *two* b's. So create them together: $S \rightarrow aSbb \mid \lambda$. (And $a^{2n}b^n$ is $S \rightarrow aaSb \mid \lambda$.)

Step 3 — “at most”: $a^n b^m$ with $n \leq m$

Now there may be *extra* b's. Matched pairs as before, plus a rule that adds a lone b: $S \rightarrow aSb \mid Sb \mid \lambda$.

Step 4 — “not equal”: $a^n b^m$ with $n \neq m$

This is the one people freeze on. The secret: **“not equal” means “more a's OR more b's”**. So build the matched part from the outside, and put **at least one** extra letter in the middle:

$$S \rightarrow aSb \mid A \mid B \quad A \rightarrow aA \mid a \quad B \rightarrow bB \mid b$$

A is “one or more a's”, B is “one or more b's”. Try $aaab$: $S \Rightarrow aSb \Rightarrow aAb \Rightarrow aaAb \Rightarrow aaab$. ✓ And ab ?

You'd need to end with A or B , which always add at least one letter, so equal counts are impossible. ✓

Step 5 — three letters: $a^n b^m c^{n+m}$

The c's must match the a's *and* the b's. Match a's with the outer c's first, then b's with the inner c's: $S \rightarrow aSc \mid B, B \rightarrow bBc \mid \lambda$.

Step 6 — mirror images: palindromes

The first and last letters must be the same: $S \rightarrow aSa \mid bSb \mid a \mid b \mid \lambda$.

Step 7 — order doesn't matter: $|w|_a = |w|_b$

Now ba and $abba$ are allowed. Idea: the first letter must be “paid back” by an opposite letter somewhere later: $S \rightarrow aSbS \mid bSaS \mid \lambda$.

Language	Grammar
$a^n b^n$	$S \rightarrow aSb \mid \lambda$
$a^n b^{2n}$	$S \rightarrow aSbb \mid \lambda$
$a^n b^m, n \leq m$	$S \rightarrow aSb \mid Sb \mid \lambda$
$a^n b^m, n < m$	$S \rightarrow aSb \mid B, B \rightarrow bB \mid b$
$a^n b^m, n \neq m$	$S \rightarrow aSb \mid A \mid B, A \rightarrow aA \mid a, B \rightarrow bB \mid b$
$a^n b^m c^{n+m}$	$S \rightarrow aSc \mid B, B \rightarrow bBc \mid \lambda$
$a^i b^j c^k, i = j \text{ or } j = k$	$S \rightarrow AC \mid B; A \rightarrow aAb \mid \lambda, C \rightarrow cC \mid \lambda; B \rightarrow aB \mid D, D \rightarrow bDc \mid \lambda$
palindromes	$S \rightarrow aSa \mid bSb \mid a \mid b \mid \lambda$
balanced brackets (a = “(”, b = “)”)	$S \rightarrow aSbS \mid \lambda$
$ w _a = w _b, \text{ any order}$	$S \rightarrow aSbS \mid bSaS \mid \lambda$

Every grammar in this table was checked by computer against its description on all strings up to length 10–12.

7.4 The real exam questions, solved slowly

“Equally many a and b, and every prefix has at least as many a as b” (10 p — asked twice)

This looks scary. It isn't. Read a as “(” and b as “)”. “Every prefix has at least as many (as)” means you never close a bracket that wasn't opened. “Equally many” means everything gets closed. So this is just **correctly balanced brackets**: $aababbab = (()())()$.

Now think about any non-empty balanced string. It starts with “(”. That bracket is closed by some “)”. Between them: a balanced string. After it: another balanced string. So:

ANSWER

$$S \rightarrow aSbS \mid \varepsilon$$

Check: $S \Rightarrow aSbS \Rightarrow abS \Rightarrow ab$. ✓ $aabb$: $S \Rightarrow aSbS \Rightarrow aaSbSbS \Rightarrow aabSbS \Rightarrow aabbS \Rightarrow aabb$. ✓ ba : every rule that produces letters starts with a , so impossible. ✓

$\{a^n b^m \mid n \neq 2m\}$ (10 p)

READ CAREFULLY — THIS EXAM CONTRADICTED ITSELF

The formula says $n \neq 2m$: the number of a's is not **twice** the number of b's. The Swedish sentence under it said the number of a's is not **half** the number of b's, which is $m \neq 2n$. If that happens to you: write one line saying which reading you answer, then answer it.

Same trick as step 4: “not equal” = “too many” or “too few”. The *equal* case is $E \rightarrow aaEb \mid \lambda$ (two a's per b).

- **Too many a's** ($n > 2m$): an equal string with at least one extra a in front: $A \rightarrow aA \mid aE$.
- **Too few a's** ($n < 2m$): each b brings 0, 1 or 2 a's — and at least one b brings fewer than 2: $B \rightarrow aaBb \mid aUb \mid Ub$, where $U \rightarrow aaUb \mid aUb \mid Ub \mid \lambda$ (“each b with at most two a's”).

ANSWER (FORMULA READING, $N \neq 2M$)

$S \rightarrow A \mid B \quad A \rightarrow aA \mid aE \quad E \rightarrow aaEb \mid \lambda \quad B \rightarrow aaBb \mid aUb \mid Ub \quad U \rightarrow aaUb \mid aUb \mid Ub \mid \lambda$

ANSWER (SENTENCE READING, $M \neq 2N$)

$S \rightarrow A \mid B \quad A \rightarrow Ab \mid Eb \quad E \rightarrow aEbb \mid \lambda \quad B \rightarrow aBbb \mid aUb \mid aU \quad U \rightarrow aUbb \mid aUb \mid aU \mid \lambda$

“Which of these strings can G generate?” (10 p)

$S \rightarrow SB \mid aA \mid cCb$, $A \rightarrow aAb \mid \varepsilon$, $B \rightarrow BB \mid C$, $C \rightarrow \varepsilon$. Nine candidate strings.

Don't start trying strings. First ask of every variable: **what can this produce?** Start from the bottom:

- C can only become ε . Nothing else.
- $B \rightarrow BB \mid C$: any number of C's — which are all nothing. So B also only gives ε .
- $A \rightarrow aAb \mid \varepsilon$: that's $a^k b^k$.
- So $S \rightarrow SB$ adds nothing, $S \rightarrow aA$ gives $a \cdot a^k b^k = a^{k+1} b^k$, and $S \rightarrow cCb$ gives just cb .

ANSWER

$L(G) = \{a^{k+1} b^k \mid k \geq 0\} \cup \{cb\} = \{a, aab, aaabb, \dots\} \cup \{cb\}$. **None** of the nine strings has that shape (ε , abcb, aabcb, cbab, abc, acb, aaabbbbccbb, aaabbbbccbbcc, cbbbaab), so G generates **none of them**.

It feels wrong to answer “none” — trust your analysis. That's the whole point of the question.

7.5 Ambiguity — when one string has two meanings

Consider $E \rightarrow E + E \mid E * E \mid a$. The string $a + a * a$ has **two** different parse trees: one where the $+$ is on top (meaning $a + (a * a)$), one where the $*$ is on top (meaning $(a + a) * a$). A grammar with such a string is **ambiguous**.

For a programming language this is a disaster: $2+3*4$ would mean both 14 and 20. How languages fix it is the subject of the next chapter. To **prove** a grammar ambiguous on the exam: give one string and two different parse trees (or two different leftmost derivations).

YOUR TURN

1. $\{a^n b^m c^n \mid n, m \geq 0\}$ 2. $\{a^n b^m \mid m \leq n \leq 2m\}$ 3. Odd-length strings whose middle letter is a .

ANSWERS

1. $S \rightarrow aSc \mid B, B \rightarrow bB \mid \lambda$ — a's wrap c's, free b's in the middle. 2. $S \rightarrow aSb \mid aaSb \mid \lambda$ — each b takes one or two a's. 3. $S \rightarrow XSX \mid a, X \rightarrow a \mid b$ — any letter on each side, the centre is a .

Where we're going next. Ambiguity isn't just a theory problem — every programming language must decide what $2 + 3 * 4$ means. That decision is called precedence, and it's built right into the grammar.

8. Precedence and associativity PL

INTUITION

Without parentheses, $2 + 3 * 4$ could mean 20 or 14. The language must decide. **Precedence**: which operator wins between *different* operators ($*$ beats $+$). **Associativity**: which side wins between *equal* operators ($10 - 4 - 3$ is $(10 - 4) - 3$).

8.1 Common rules

Level (high → low)	Operators	Associativity
1	$\wedge$ / $**$ (power)	right: $2^3^2 = 2^3^2 = 512$
2	unary $-$, $!$	right
3	$*$ / $\%$	left
4	$+$ -	left: $10-4-3 = 3$
5	$<$ $==$...	often non-associative
6	$\&\&$, then $ $	left
7	$=$ (assignment)	right: $a = b = 0$

In Haskell you can even declare them: `infixl 6 +`, `infixl 7 *`, `infixr 8 ^`, `infixr 5 :`, `infixr 5 ++`. Function application binds tighter than any operator: `f x + 1 = (f x) + 1`.

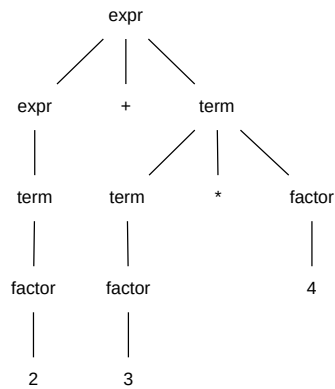
8.2 How a grammar encodes them

RECIPE: ONE VARIABLE PER PRECEDENCE LEVEL

```
expr  → expr + term  | expr - term  | term          (lowest precedence, left-assoc)
term  → term * factor | term / factor | factor
factor → base ^ factor | base          (right-assoc: RIGHT recursion)
base  → number | ( expr )              (highest: parentheses)
```

- **Lower in the grammar = binds tighter**: a `term` is built before it can be added.
- **Left recursion** (`expr → expr + term`) grows the tree to the left $\Rightarrow$ **left associative**.
- **Right recursion** (`factor → base ^ factor`) $\Rightarrow$ **right associative**.

This grammar is unambiguous: $2 + 3 * 4$ has exactly one tree, with $*$ below $+$:



A working parser built exactly this way (in Haskell) is in Chapter 22; it really turns `10 - 4 - 3` into `(10 - 4) - 3`.

MODEL ANSWER (6 P): “BRIEFLY DESCRIBE PRECEDENCE AND ASSOCIATIVITY. HOW ARE THEY RELATED?”

Precedence decides which of two **different** operators binds tighter when there are no parentheses: `*` has higher precedence than `+`, so `2 + 3 * 4 = 2 + (3 * 4) = 14`. **Associativity** decides how operators of the **same** precedence level are grouped: `-` is left-associative, so `10 - 4 - 3 = (10 - 4) - 3 = 3`; exponentiation and assignment are usually right-associative (`2^3^2 = 2^(3^2)`, `a = b = 0`).

Relation: both are rules that remove the **ambiguity** of expressions without parentheses and thereby fix the shape of the syntax tree (and so the result). Precedence is used between different levels; associativity only matters when operators have the **same** precedence. In a grammar they are encoded by one non-terminal per precedence level (lower levels deeper in the tree bind tighter) and by the direction of recursion (left recursion $\Rightarrow$ left associative, right recursion $\Rightarrow$ right associative). Parentheses override both.

9. Giving the machine a stack: pushdown automata FCS

Grammars *describe* nested languages. What *machine* recognises them? Let's go back to our door-keeper with sticky notes, who failed at $a^n b^n$ because they couldn't count. Give them one extra tool: **a stack of plates**. They can put a plate on top, take the top plate off, and look at the top plate — nothing else. No peeking lower down.

Now $a^n b^n$ is easy: for every a , put a plate on the stack. For every b , take one off. If the stack is back to empty exactly when the string ends, the counts matched. And brackets? Put a plate on for “(”, take one off for “)”. A stack is *exactly* the memory that nesting needs: the last thing opened is the first thing closed.

DEFINITION — NPDA (ON YOUR EXAM SHEET)

$M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$: states, input alphabet, **stack alphabet** Γ (the kinds of plates), transitions δ , start state, a **bottom-of-stack** symbol z , accepting states.

A transition $(p, x) \in \delta(q, a, s)$ reads: “in state q , reading a (or nothing, if $a = \lambda$), with s on top of the stack — go to p and replace s by the string x ”. So $x = \lambda$ **pops**, $x = As$ **pushes** A , and $x = s$ leaves the stack alone.

9.1 The machine for $a^n b^n$

Transition	In words
$\delta(q_0, a, z) = \{(q_0, Az)\}$ and $\delta(q_0, a, A) = \{(q_0, AA)\}$	for each a , push a plate A
$\delta(q_0, b, A) = \{(q_1, \lambda)\}$, $\delta(q_1, b, A) = \{(q_1, \lambda)\}$	for each b , pop a plate; after the first b we're in q_1 , so no more a 's are allowed
$\delta(q_0, \lambda, z) = \{(q_2, z)\}$, $\delta(q_1, \lambda, z) = \{(q_2, z)\}$	back at the bottom: go to the accepting state q_2

Watch it run on $aabb$. We write (state, what's left to read, stack):

$$(q_0, aabb, z) \vdash (q_0, abb, Az) \vdash (q_0, bb, AAz) \vdash (q_1, b, Az) \vdash (q_1, \lambda, z) \vdash (q_2, \lambda, z) \checkmark$$

Those snapshots are called **instantaneous descriptions**. The exam likes them because they show you understand how the machine moves.

9.2 Why the N matters here

For palindromes ww^R , the idea is: push the first half, then pop and compare the second half. But when does the second half start? The machine can't know — so it **guesses**, at every position, “maybe this is the middle”. One of those guesses is right.

With finite automata, guessing gave no extra power (Chapter 2). With stacks, it **does**: palindromes can be recognised by a guessing PDA but by no deterministic one. So:

$$\text{regular} \subset \text{deterministic context-free (DPDA)} \subset \text{context-free (NPDA)}$$

$a^n b^n$ is in the middle class (a DPDA can do it — no guessing needed), ww^R is only in the outer one. Programming-language parsers use deterministic grammars, because a parser that must guess

would be too slow.

9.3 Grammars and stack machines are the same power

THEOREM

A language is generated by a context-free grammar **if and only if** it is accepted by an NPDA.

The direction grammar $\rightarrow$ machine is lovely and worth understanding: put S on the stack. If the top is a variable, replace it by the right side of one of its rules (guessing which). If the top is a letter, it must match the next input letter — pop it. If the stack empties exactly as the input ends, accept. The stack holds “what I still expect to see”, and the machine is simply acting out a leftmost derivation. **That is precisely what a top-down parser does.**

Where we’re going next. Grammars can be messy — λ -rules, rules that just rename, rules with long right sides. For a computer to parse efficiently, it helps to first rewrite the grammar into a tidy standard shape. That’s Chomsky normal form — and with it, a neat algorithm (CYK) that decides whether a string belongs to the language.

10. Chomsky normal form and the CYK algorithm BOTH

Last chapter ended with a promise: tidy the grammar up, and a simple algorithm can decide membership. That's this chapter, and it's probably the most **mechanical** one in the whole guide — which is great news. There is nothing to invent here. If you follow the steps, you get full points. So let's learn the steps so well you could do them half asleep.

Part A — Chomsky normal form

THE IDEA IN PLAIN WORDS

A grammar in CNF is a grammar where **every rule is one of only two boring shapes**:

$$A \rightarrow BC \text{ (exactly two variables) or } A \rightarrow a \text{ (exactly one terminal).}$$

Why would anyone want that? Because then every derivation tree is a **binary tree**, and a string of length n is always derived in exactly $2n - 1$ steps. That makes algorithms like CYK (Part B) possible. Converting to CNF does **not change the language** (except that λ disappears) — it only rewrites the rules into boring shapes.

DEFINITION (WRITE THIS ON THE EXAM)

A context-free grammar is in **Chomsky normal form** if every production has the form

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a$$

where A, B, C are variables and a is a terminal. Every CFG G with $\lambda \notin L(G)$ has an equivalent grammar in CNF. (If $\lambda \in L(G)$, the CNF grammar generates $L(G) \setminus \{\lambda\}$.)

Checking whether a grammar is in CNF

Go through the rules one by one and mark anything that is not of those two shapes:

Rule	CNF?	Why
$S \rightarrow AB$	✓	two variables
$A \rightarrow a$	✓	one terminal
$S \rightarrow aB$	✗	terminal mixed with a variable
$S \rightarrow ABC$	✗	three variables (too long)
$A \rightarrow B$	✗	a <i>unit</i> production (single variable)
$A \rightarrow \lambda$	✗	a λ -production
$A \rightarrow ab$	✗	two terminals

10.A1 The recipe (always in this order: $\lambda \rightarrow$ units $\rightarrow$ useless $\rightarrow$ shape)

RECIPE: CONVERTING TO CNF

1. Remove λ -productions.

- Find the **nullable** variables: A is nullable if $A \rightarrow \lambda$, or $A \rightarrow B_1 \dots B_k$ where all B_i are nullable. Repeat until nothing new.
- For every rule, add **all versions where you leave out any combination of nullable variables** (but never add $A \rightarrow \lambda$).
- Delete all $A \rightarrow \lambda$.

2. Remove unit productions ($A \rightarrow B$).

- For every A , find all B with $A \Rightarrow^* B$ using *only* unit rules.
- Give A a copy of every **non-unit** rule of each such B .
- Delete all unit rules (including $A \rightarrow A$).

3. Remove useless variables, in this order:

- non-generating**: variables that can never finish into a terminal string (e.g. $C \rightarrow cC$ only);
- non-reachable**: variables that can't be reached from S .

Delete every rule that mentions them.

4. Fix the shapes.

- In every rule whose right side has length ≥ 2 , replace each terminal a by a new variable T_a and add $T_a \rightarrow a$.
- Break every right side with 3 or more variables into a chain: $A \rightarrow C_1 C_2 C_3 \dots C_n$ becomes $A \rightarrow C_1 D_1, D_1 \rightarrow C_2 D_2, \dots, D_{n-2} \rightarrow C_{n-1} C_n$.

WHY THE ORDER MATTERS

Removing λ can **create** unit rules (e.g. $S \rightarrow aSb \mid SS$ with nullable S gives $S \rightarrow S$). Removing units can **create** useless variables (a variable nobody uses any more). So: λ first, units second, useless third, shapes last. If you do it in another order you will have to repeat steps.

10.A2 Worked example A — the exam's own grammar $S \rightarrow aSbS \mid \lambda$

This is the grammar for the PL-exam language “equally many a and b , every prefix has at least as many a as b ”. Convert it to CNF.

Step 1 — λ -productions. Nullable: S (because $S \rightarrow \lambda$). The rule $S \rightarrow aSbS$ contains two occurrences of S , so we write all four combinations of keeping/dropping them:

keep 1st S?	keep 2nd S?	new rule
yes	yes	$S \rightarrow aSbS$
no	yes	$S \rightarrow abS$
yes	no	$S \rightarrow aSb$
no	no	$S \rightarrow ab$

Delete $S \rightarrow \lambda$. Result: $S \rightarrow aSbS \mid abS \mid aSb \mid ab$.

Step 2 — unit productions. None (no rule of the form $S \rightarrow X$). **Step 3 — useless.** None.

Step 4a — terminals. Introduce $A \rightarrow a$ and $B \rightarrow b$: $S \rightarrow ASBS \mid ABS \mid ASB \mid AB$.

Step 4b — long right sides.

- $S \rightarrow ASBS$ (4 symbols) $\rightarrow S \rightarrow AX$, $X \rightarrow SW$, $W \rightarrow BS$.
- $S \rightarrow ABS$ (3 symbols) $\rightarrow S \rightarrow AY$, $Y \rightarrow BS$.
- $S \rightarrow ASB$ (3 symbols) $\rightarrow S \rightarrow AZ$, $Z \rightarrow SB$.
- $S \rightarrow AB$ is already fine.

FINAL ANSWER

$S \rightarrow AX \mid AY \mid AZ \mid AB$ $X \rightarrow SW$ $W \rightarrow BS$ $Y \rightarrow BS$ $Z \rightarrow SB$ $A \rightarrow a$ $B \rightarrow b$

Since λ was in the original language, the CNF grammar generates the same language minus λ . (You may reuse W instead of introducing Y — both are correct.)

10.A3 Worked example B — every step is needed

G : $S \rightarrow ASA \mid aB$ $A \rightarrow B \mid S$ $B \rightarrow b \mid \lambda$

Step 1 — λ -productions.

- Nullable: B (since $B \rightarrow \lambda$); then A (since $A \rightarrow B$ and B is nullable). Is S ? $S \rightarrow ASA$ needs S itself nullable, $S \rightarrow aB$ has a terminal. So **no**. Nullable = $\{A, B\}$.
- $S \rightarrow ASA$: the two A 's may each be dropped: ASA , SA , AS , S .
- $S \rightarrow aB$: drop B : aB , a .
- $A \rightarrow B$: dropping B would give $A \rightarrow \lambda$ — never add that. $A \rightarrow S$ unchanged.
- Delete $B \rightarrow \lambda$.

Result: $S \rightarrow ASA \mid SA \mid AS \mid S \mid aB \mid a$ $A \rightarrow B \mid S$ $B \rightarrow b$

Step 2 — unit productions. The unit rules are $S \rightarrow S$, $A \rightarrow B$, $A \rightarrow S$.

- $S \rightarrow S$: just delete it.
- $A \Rightarrow^* B$ and $A \Rightarrow^* S$. So A gets all non-unit rules of B (b) and of S (ASA , SA , AS , aB , a).

Result: $S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$ $A \rightarrow b \mid ASA \mid SA \mid AS \mid aB \mid a$ $B \rightarrow b$

Step 3 — useless. Everything generates a terminal string and is reachable. Nothing to remove.

Step 4. $aB \rightarrow UB$ with $U \rightarrow a$. $ASA \rightarrow AX$ with $X \rightarrow SA$.

FINAL ANSWER

$S \rightarrow AX \mid SA \mid AS \mid UB \mid a$
 $A \rightarrow b \mid AX \mid SA \mid AS \mid UB \mid a$
 $B \rightarrow b$ $U \rightarrow a$ $X \rightarrow SA$

10.A4 Worked example C — unit chains and useless variables

G : $S \rightarrow Aa \mid B \mid C$ $A \rightarrow a \mid bA$ $B \rightarrow A \mid bb$ $C \rightarrow cC$

1. λ : no λ -rules. Skip.

2. **Units:** $S \rightarrow B$, $S \rightarrow C$, $B \rightarrow A$. Unit reachability: $S \Rightarrow^* B, A, C$; $B \Rightarrow^* A$. So S gets bb (from B), a, bA (from A), cC (from C); B gets a, bA . Now: $S \rightarrow Aa \mid bb \mid a \mid bA \mid cC$, $A \rightarrow a \mid bA$, $B \rightarrow bb \mid a \mid bA$, $C \rightarrow$

cC .

3. **Useless:** (a) C is **not generating** — $C \rightarrow cC$ never ends. Delete C and $S \rightarrow cC$. (b) B is now **unreachable** — nobody mentions B . Delete it. Now: $S \rightarrow Aa \mid bb \mid a \mid bA$, $A \rightarrow a \mid bA$.
4. **Shapes:** $V \rightarrow a$ and $T \rightarrow b$: $S \rightarrow AV \mid TT \mid a \mid TA$, $A \rightarrow a \mid TA$.

FINAL ANSWER

$S \rightarrow AV \mid TT \mid a \mid TA$ $A \rightarrow a \mid TA$ $V \rightarrow a$ $T \rightarrow b$

Note: you cannot reuse A for “ $\rightarrow a$ ” here, because A also has the rule $A \rightarrow TA$; a new variable that *only* produces a is required.

THE MISTAKES THAT COST POINTS

- Dropping **only one** combination of nullable symbols. With two nullable symbols in a rule there are up to 4 versions (with three, up to 8). Make the little table.
- Adding $A \rightarrow \lambda$ back by dropping everything.
- Forgetting that a variable is nullable *indirectly* ($A \rightarrow B$, $B \rightarrow \lambda$).
- Removing a unit rule without copying the rules over (then the language shrinks).
- Replacing a terminal by a variable that has other rules too (see example C).
- Not saying anything about λ . One sentence: “ $\lambda \in L(G)$, so the CNF grammar generates $L(G) - \{\lambda\}$.”

Part B — The CYK algorithm

THE IDEA IN PLAIN WORDS

Question: *can the grammar produce the string w ?* CYK answers it bottom-up, like building a pyramid:

- First, for every **single letter**: which variables can produce it? (Look for rules $A \rightarrow a$.)
- Then, for every piece of **length 2**: which variables can produce it? Because the grammar is in CNF, a variable produces a piece only through a rule $A \rightarrow BC$: B makes the left part, C makes the right part. So **try every place to cut the piece in two**, and look up the two halves in the rows you already have.
- Then length 3, 4, ... up to the whole string.
- At the top: if the **start symbol** can produce the whole string, $w \in L(G)$.

That's all. The grammar **must be in CNF** first (that's why CNF and CYK always come together).

DEFINITION — WRITE THIS ON THE EXAM

Let $w = a_1 a_2 \cdots a_n$. Define V_{ij} = the set of variables that derive the substring $a_i \cdots a_j$. Then

$$V_{ii} = \{A \mid A \rightarrow a_i\}$$
$$V_{ij} = \bigcup_{k=i}^{j-1} \{A \mid A \rightarrow BC, B \in V_{ik}, C \in V_{k+1,j}\}$$

and $w \in L(G) \iff S \in V_{1n}$. The running time is $O(n^3)$.

RECIPE FOR THE EXAM

1. Check that the grammar is in CNF (say so explicitly — the exam often asks).
2. Draw a triangle: row 1 has n cells (one per letter), row 2 has $n - 1$, ..., the top row has one cell. Label each cell with its substring — this avoids mistakes.
3. Row 1: for each letter, write the variables with a rule $X \rightarrow$ that letter.
4. Row ℓ : for each substring of length ℓ , for **every** cut into (left, right): take the pairs $B \in \text{cell}(\text{left})$, $C \in \text{cell}(\text{right})$, and add every A with a rule $A \rightarrow BC$. Write $\emptyset$ if nothing.
5. Top cell: contains S ? Then write the conclusion sentence (below).

THE “VERY BRIEFLY EXPLAIN WHAT THE RESULT MEANS” SENTENCE

“Cell V_{ij} contains exactly the variables that can derive the substring from position i to j . Since the start symbol S is in the top cell V_{1n} , $S \Rightarrow^* w$, so the string **is** in the language of the grammar.” (If $S \notin V_{1n}$: “...so the string is **not** in the language.”)

Bonus sentence for full points: “Following the entries back down (which cut produced S) gives a derivation tree.”

10.B1 Worked example 1 — the real exam question (abcab)

Grammar: $S \rightarrow SS \mid XY \mid XS', S' \rightarrow SY, X \rightarrow a, Y \rightarrow BC \mid BB \mid b, B \rightarrow b, C \rightarrow CC \mid c$.

Is it in CNF? Yes: every rule is either two variables ($SS, XY, XS', SY, BC, BB, CC$) or one terminal (a, b, c). (S' is just a variable name.) So the answer is “Yes”.

Here is every cell with every cut tried — exactly the work you do on paper:

```
V11 ('a'): variables with a rule  $\rightarrow a$ : {X}
V22 ('b'): variables with a rule  $\rightarrow b$ : {B, Y}
V33 ('c'): variables with a rule  $\rightarrow c$ : {C}
V44 ('a'): variables with a rule  $\rightarrow a$ : {X}
V55 ('b'): variables with a rule  $\rightarrow b$ : {B, Y}

V12 ('ab'): a|b: {X} · {B, Y}  $\rightarrow$  {S}  $\Rightarrow$  {S}
V23 ('bc'): b|c: {B, Y} · {C}  $\rightarrow$  {Y}  $\Rightarrow$  {Y}
V34 ('ca'): c|a: {C} · {X}  $\rightarrow \emptyset \Rightarrow \emptyset$ 
V45 ('ab'): a|b: {X} · {B, Y}  $\rightarrow$  {S}  $\Rightarrow$  {S}

V13 ('abc'): a|bc: {X} · {Y}  $\rightarrow$  {S}; ab|c: {S} · {C}  $\rightarrow \emptyset \Rightarrow$  {S}
V24 ('bca'): b|ca: {B, Y} ·  $\emptyset \rightarrow \emptyset$ ; bc|a: {Y} · {X}  $\rightarrow \emptyset \Rightarrow \emptyset$ 
V35 ('cab'): c|ab: {C} · {S}  $\rightarrow \emptyset$ ; ca|b:  $\emptyset$  · {B, Y}  $\rightarrow \emptyset \Rightarrow \emptyset$ 

V14 ('abca'): a|bca: {X} ·  $\emptyset \rightarrow \emptyset$ ; ab|ca: {S} ·  $\emptyset \rightarrow \emptyset$ ; abc|a: {S} · {X}  $\rightarrow \emptyset \Rightarrow \emptyset$ 
V25 ('bcab'): b|cab: {B, Y} ·  $\emptyset \rightarrow \emptyset$ ; bc|ab: {Y} · {S}  $\rightarrow \emptyset$ ; bca|b:  $\emptyset$  · {B, Y}  $\rightarrow \emptyset \Rightarrow \emptyset$ 

V15 ('abcab'): a|bcab: {X} ·  $\emptyset \rightarrow \emptyset$ ; ab|cab: {S} ·  $\emptyset \rightarrow \emptyset$ ; abc|ab: {S} · {S}  $\rightarrow$  {S}; abca|b:  $\emptyset$  · {B, Y}  $\rightarrow \emptyset \Rightarrow$  {S}

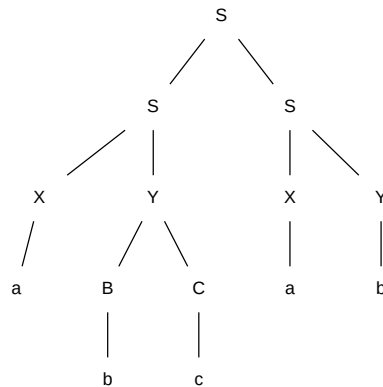
Start symbol  $S \in V15 \Rightarrow$  'abcab' IS in  $L(G)$ 
```

How to read one line: $abc|ab: \{S\} \cdot \{S\} \rightarrow \{S\}$ means “cut *abcab* into *abc* and *ab*; S makes *abc*, S makes *ab*; the rule $S \rightarrow SS$ combines them, so S makes *abcab*”.

The table as you would draw it on the exam (bottom row = single letters):

len 5	$V_{15} = \{S\}$				
len 4	$V_{14} = \emptyset$	$V_{25} = \emptyset$			
len 3	$V_{13} = \{S\}$	$V_{24} = \emptyset$	$V_{35} = \emptyset$		
len 2	$V_{12} = \{S\}$	$V_{23} = \{Y\}$	$V_{34} = \emptyset$	$V_{45} = \{S\}$	
len 1	$V_{11} = \{X\}$	$V_{22} = \{B, Y\}$	$V_{33} = \{C\}$	$V_{44} = \{X\}$	$V_{55} = \{B, Y\}$
	a	b	c	a	b

Result: $S \in V_{15}$, so **abcab** $\in L(G)$. Reading back: $S \rightarrow SS$ with the first S deriving abc (via $S \rightarrow XY$, $X \rightarrow a, Y \rightarrow BC \rightarrow bc$) and the second deriving ab (via $S \rightarrow XY$, $X \rightarrow a, Y \rightarrow b$).



Derivation tree recovered from the table

10.B2 Worked example 2 — a string that is *rejected* (length 5)

Grammar: the CNF grammar from Part A (for the language “balanced a/b, prefixes have $\geq$ as many a as b”): $S \rightarrow AX \mid AY \mid AZ \mid AB, X \rightarrow SW, W \rightarrow BS, Y \rightarrow BS, Z \rightarrow SB, A \rightarrow a, B \rightarrow b$. String: **aabab**.

```

V11 ('a'): variables with a rule  $\rightarrow a$ : {A}
V22 ('a'): variables with a rule  $\rightarrow a$ : {A}
V33 ('b'): variables with a rule  $\rightarrow b$ : {B}
V44 ('a'): variables with a rule  $\rightarrow a$ : {A}
V55 ('b'): variables with a rule  $\rightarrow b$ : {B}

V12 ('aa'): a|a: {A}·{A}  $\rightarrow \emptyset \Rightarrow \emptyset$ 
V23 ('ab'): a|b: {A}·{B}  $\rightarrow \{S\} \Rightarrow \{S\}$ 
V34 ('ba'): b|a: {B}·{A}  $\rightarrow \emptyset \Rightarrow \emptyset$ 
V45 ('ab'): a|b: {A}·{B}  $\rightarrow \{S\} \Rightarrow \{S\}$ 

V13 ('aab'): a|ab: {A}·{S}  $\rightarrow \emptyset$ ; aa|b:  $\emptyset$ ·{B}  $\rightarrow \emptyset \Rightarrow \emptyset$ 
V24 ('aba'): a|ba: {A}· $\emptyset \rightarrow \emptyset$ ; ab|a: {S}·{A}  $\rightarrow \emptyset \Rightarrow \emptyset$ 
V35 ('bab'): b|ab: {B}·{S}  $\rightarrow \{W, Y\}$ ; ba|b:  $\emptyset$ ·{B}  $\rightarrow \emptyset \Rightarrow \{W, Y\}$ 

V14 ('aaba'): a|aba: {A}· $\emptyset \rightarrow \emptyset$ ; aa|ba:  $\emptyset$ · $\emptyset \rightarrow \emptyset$ ; aab|a:  $\emptyset$ ·{A}  $\rightarrow \emptyset \Rightarrow \emptyset$ 
V25 ('abab'): a|bab: {A}·{W, Y}  $\rightarrow \{S\}$ ; ab|ab: {S}·{S}  $\rightarrow \emptyset$ ; aba|b:  $\emptyset$ ·{B}  $\rightarrow \emptyset \Rightarrow \{S\}$ 

V15 ('aabab'): a|abab: {A}·{S}  $\rightarrow \emptyset$ ; aa|bab:  $\emptyset$ ·{W, Y}  $\rightarrow \emptyset$ ; aab|ab:  $\emptyset$ ·{S}  $\rightarrow \emptyset$ ; aaba|b:  $\emptyset$ ·{B}  $\rightarrow \emptyset \Rightarrow \emptyset$ 

Start symbol  $S \notin V_{15} \Rightarrow$  'aabab' is NOT in  $L(G)$ 

```

$S \notin V_{15} = \emptyset$, so **aabab** is **not** in the language. Sanity check: it has 3 a's and 2 b's, so it can't be balanced.
✓ (Always do such a sanity check if you can!)

10.B3 Worked example 3 — accepted, length 6

```

V11 ('a'): variables with a rule → a: {A}
V22 ('b'): variables with a rule → b: {B}
V33 ('a'): variables with a rule → a: {A}
V44 ('a'): variables with a rule → a: {A}
V55 ('b'): variables with a rule → b: {B}
V66 ('b'): variables with a rule → b: {B}

V12 ('ab'): a|b: {A}·{B} → {S} ⇒ {S}
V23 ('ba'): b|a: {B}·{A} → ∅ ⇒ ∅
V34 ('aa'): a|a: {A}·{A} → ∅ ⇒ ∅
V45 ('ab'): a|b: {A}·{B} → {S} ⇒ {S}
V56 ('bb'): b|b: {B}·{B} → ∅ ⇒ ∅

V13 ('aba'): a|ba: {A}·∅ → ∅; ab|a: {S}·{A} → ∅ ⇒ ∅
V24 ('baa'): b|aa: {B}·∅ → ∅; ba|a: ∅·{A} → ∅ ⇒ ∅
V35 ('aab'): a|ab: {A}·{S} → ∅; aa|b: ∅·{B} → ∅ ⇒ ∅
V46 ('abb'): a|bb: {A}·∅ → ∅; ab|b: {S}·{B} → {Z} ⇒ {Z}

V14 ('abaa'): a|baa: {A}·∅ → ∅; ab|aa: {S}·∅ → ∅; aba|a: ∅·{A} → ∅ ⇒ ∅
V25 ('baab'): b|aab: {B}·∅ → ∅; ba|ab: ∅·{S} → ∅; baa|b: ∅·{B} → ∅ ⇒ ∅
V36 ('aabb'): a|abb: {A}·{Z} → {S}; aa|bb: ∅·∅ → ∅; aab|b: ∅·{B} → ∅ ⇒ {S}

V15 ('abaab'): a|baab: {A}·∅ → ∅; ab|aab: {S}·∅ → ∅; aba|ab: ∅·{S} → ∅; abaa|b: ∅·{B} → ∅ ⇒ ∅
V26 ('baabb'): b|aabb: {B}·{S} → {W,Y}; ba|abb: ∅·{Z} → ∅; baa|bb: ∅·∅ → ∅; baab|b: ∅·{B} → ∅ ⇒ {W,Y}

V16 ('abaabb'): a|baabb: {A}·{W,Y} → {S}; ab|aabb: {S}·{S} → ∅; aba|abb: ∅·{Z} → ∅; abaa|bb: ∅·∅ → ∅; abaab|b: ∅·{B} → ∅ ⇒ {S}

Start symbol S ∈ V16 ⇒ 'abaabb' IS in L(G)

```

$S \in V_{16}$: abaabb is in the language (it is $ab \cdot aabb$).

THE MISTAKES THAT COST POINTS

- **Not trying every cut.** A cell of length ℓ has $\ell - 1$ cuts. Example 1's top cell has 4 cuts, and only the third one works.
- Order matters: $A \rightarrow BC$ needs B on the **left** part and C on the **right** part.
- Forgetting that a cell can hold **several** variables (like $\{B, Y\}$) — try all of them.
- Not writing the conclusion sentence. “Briefly explain what the result means” is worth points on its own.
- Running CYK on a grammar that is not in CNF — convert first (Part A).

11. Pumping lemma for CFLs, and closure FCS

INTUITION

In a very tall parse tree, some variable must repeat on one path (finitely many variables — pigeonhole again). The subtree between the two occurrences can be repeated or removed. That pumps **two** pieces u and v **at the same time** (one on each side of the middle y). This is why CFLs can match **two** counts ($a^n b^n$) but not **three** ($a^n b^n c^n$).

THE LEMMA (EXAM APPENDIX)

If L is (infinite) context-free, there is n such that every $w \in L$ with $|w| \geq n$ can be written $w = xuyvz$ with $|uyv| \leq n$, $|uv| \geq 1$, such that $xu^i y v^i z \in L$ for all $i \geq 0$.

RECIPE: PROVING L IS NOT CONTEXT-FREE

1. Assume L is CF with constant n .
2. Choose $w \in L$, $|w| \geq n$, depending on n — usually blocks of length n : $a^n b^n c^n$.
3. The window uyv has length $\leq n$, so it **can touch at most two neighbouring blocks** — but it can be **anywhere** (unlike the regular lemma). List the cases by which blocks u and v touch.
4. In each case choose i (0 or 2) that breaks membership.
5. All cases lead to contradiction $\Rightarrow$ not CF.

Example 1: $\{a^k b^k c^k\}$

Assume CF, constant n . $w = a^n b^n c^n$. Since $|uyv| \leq n$, uv can't contain both a 's and c 's (the b -block of length n is between them). Case 1: uv has no c . Then pumping $i = 0$ removes at least one a or b but no c : the counts are no longer equal. Case 2: uv has no a : symmetric, $i = 0$ leaves n a 's and fewer b 's or c 's. (Also if u or v contains two different letters, $i = 2$ puts letters out of order.) Contradiction — not CF.

Example 2: $\{ww \mid w \in \{a, b\}^*\}$

$w = a^n b^n a^n b^n$. The window touches at most two adjacent blocks. $i = 0$: if the window is in the first half, the first half loses letters but the second doesn't; the string (if even length) is no longer of the form ww because its middle shifts into the b -blocks mismatching. Case analysis (first half / middle / second half) gives a contradiction each time. Not CF. (Note: $\{ww^R\}$ is CF — mirror, not copy.)

Example 3: $\{a^k \mid k \text{ prime}\}, \{a^{k^2}\}$

Over one letter, CF = regular, so these are not CF either (same proofs as Ch. 6 with $|uv|$ in place of $|y|$).

FCS EXAM Q4 (2022/2025 COVER): THE LANGUAGE PRINTED IS ACTUALLY CONTEXT-FREE

$L = \{w \mid |w|_a > 2|w|_b\} \cup \{w \mid 2|w|_a < |w|_b\}$. Each part is recognised by a PDA that uses its stack as a counter (add 1 per a , subtract 2 per b ; accept if the counter is positive), and CFLs are closed under union. So L is context-free, and no correct proof of the opposite exists. A grammar for it:

$S \rightarrow A \mid B, A \rightarrow E a E \mid E a A, B \rightarrow F b F \mid F b B, E \rightarrow a E a E b E \mid a E b E a E \mid b E a E a E \mid \lambda, F \rightarrow b F b F a F \mid b F a F b F \mid a F b F b F \mid \lambda$

If this happens on your exam: don't freeze. Write briefly why the attempt fails, give the PDA/union argument, and say you believe the question contains a mistake — then move on. If a similar question appears with three letters, e.g. $\{w \in \{a, b, c\}^* \mid |w|_a = |w|_b = |w|_c\}$, it is non-CF: intersect with the regular $a^*b^*c^*$ to get $a^n b^n c^n$ (CFL $\cap$ regular is CF, so if L were CF, $a^n b^n c^n$ would be too).

11.1 Closure properties of CFLs

Operation	CFL	DCFL	Why
Union	yes	no	$S \rightarrow S_1 \mid S_2$
Concatenation	yes	no	$S \rightarrow S_1 S_2$
Star	yes	no	$S \rightarrow S_1 S \mid \lambda$
Intersection	no	no	$\{a^n b^n c^m\} \cap \{a^m b^n c^n\} = \{a^n b^n c^n\}$
Complement	no	yes	if CFL were closed under complement, then $\cap$ via De Morgan
$\cap$ with a regular language	yes	yes	product of PDA and DFA (one stack is enough)

Decidable for CFLs: membership (CYK), emptiness, finiteness. **Undecidable:** whether two CFGs are equivalent, whether a CFG is ambiguous, whether $L(G) = \Sigma^*$.

Part II

The ultimate machine — and what even it cannot do



12. Turing machines FCS

Our door-keeper started with sticky notes (finite automata), then got a stack of plates (pushdown automata). Each upgrade let them recognise more. Now let's go all the way. Give them an **endless notebook**: they can write anywhere, erase, flip back and forth, reread. That's a **Turing machine** — and here is the astonishing part: **no further upgrade helps**. Everything any computer can compute, a Turing machine can compute. Your laptop is, in theory, just a fast Turing machine.

12.1 How it works

THE PICTURE

An infinite tape of cells. Your input is written on it, blanks ($\square$) everywhere else. A **head** sits on one cell. At each step, looking only at its current state and the symbol under the head, the machine: **writes** a symbol, **moves** one cell left or right, and **changes state**. When there is no rule for the current situation, it **halts**. If it halts in an accepting state, the input is accepted.

Two big differences from before: it can **go back**, and it might **never stop**.

DEFINITION (ON YOUR EXAM SHEET)

$M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ with tape alphabet $\Gamma \supseteq \Sigma \cup \{\square\}$ and $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$.
 $\delta(q, a) = (p, b, R)$ means: in state q reading a , write b , move right, go to state p .

12.2 Watching one work: $a^n b^n$

A stack machine did this by counting. A Turing machine does it the way you'd do it with a pencil: **cross off** an a , run over to the first uncrossed b , cross it off, run back, repeat. If everything gets crossed off in pairs, accept.

State	Its job
q_0	"Find the next a ." Reading a : replace it by x , move right, go to q_1 . Reading y instead: no a 's left — go to q_3 to check.
q_1	"Run right to the first b ." Skip over a 's and y 's. Reading b : replace by y , move left, go to q_2 .
q_2	"Run back left." Skip over y 's and a 's. Reading x : step right, back to q_0 .
q_3	"Make sure only y 's remain." Skip y 's. Reading $\square$: accept (go to q_4).

On input $aabb$, writing the state just before the symbol the head is on:

$q_0aabb \vdash xq_1abb \vdash xaq_1bb \vdash xq_2ayb \vdash q_2xayb \vdash xq_0ayb \vdash xxq_1yb \vdash xxyq_1b \vdash xxq_2yy \vdash xq_2xyy \vdash$
 $xxq_0yy \vdash xxyq_3y \vdash xxyyq_3\square \vdash xxyyq_4\square$

Follow the x 's and y 's appear in pairs. A stuck situation (e.g. a b with no a left) has no rule, so the machine halts in a non-accepting state: reject.

12.3 Machines that compute answers — and plugging them together

A TM can also **compute a function**: start with input v on the tape, halt with $f(v)$ on the tape. Now a lovely exam question: given a machine for f and one for g , build one for $g(f(v))$. Think of it like piping programs: run the first, and hand its output to the second.

EXAM (6 P): CONSTRUCT M_{GF} FROM M_F AND M_G

Rename states so Q_f and Q_g don't overlap, and use the union of the tape alphabets. Let

$$M_{gf} = (Q_f \cup Q_g \cup \{h\}, \Sigma, \Gamma, \delta, q_{0,f}, \square, F_g)$$

- **Phase 1**: for states of M_f , δ is δ_f . So M_{gf} first behaves exactly like M_f , leaving $f(v)$ on the tape.
- **Hand-over**: wherever M_f would halt in a final state $q \in F_f$ reading a , add $\delta(q, a) = (h, a, R)$ and $\delta(h, b) = (q_{0,g}, b, L)$ for every b — a step right and back (the tape is unchanged; this just switches to M_g 's start state with the head where M_g expects it).
- **Phase 2**: for states of M_g , δ is δ_g . M_g now runs on $f(v)$ and leaves $g(f(v))$.
- Accepting states: only F_g (the old F_f states are no longer final).

State your assumption: each machine halts with its head on the first symbol of its output and nothing else on the tape.

12.4 Upgrades that don't upgrade

People have tried to make Turing machines stronger: more tapes, a 2-D tape, a “stay” move, guessing. None of it lets you compute anything new — each can be simulated by the plain machine (usually more slowly). Two tapes? Put them on one tape as two *tracks*, and mark where each head is. The exam asks about the most interesting one:

EXAM (6 P): WHY NONDETERMINISTIC TMS ARE NO MORE POWERFUL THAN DETERMINISTIC ONES

A deterministic TM is a special NTM, so NTMs are at least as powerful. For the other direction, let N be an NTM. On input w , all possible computations of N form a **tree**: each node is a configuration, and its children are the configurations reachable in one step (at most k of them, since δ offers finitely many choices). N accepts iff some node is an accepting configuration.

A deterministic machine D searches this tree **breadth-first**: for $\ell = 1, 2, 3, \dots$ it tries every sequence of at most ℓ choices, simulating N along it (convenient with three tapes: input, work, current choice sequence; and multitape machines equal single-tape ones). If N has an accepting computation of length ℓ , D finds it at round ℓ and accepts; otherwise D never accepts. Breadth-first matters: depth-first could get lost forever in one infinite branch. So $L(D) = L(N)$ — same power, though the simulation can take exponentially longer.

12.5 The universal machine

A Turing machine is described by a finite list of rules, which can be written as a string. So one machine, U , can take *another machine's description* plus an input, and simulate it. That's a **universal Turing machine** — it's the idea of a computer that runs programs, invented before computers existed. It also means: there are only **countably many** Turing machines (they're strings). Keep that in mind for the next chapter.

12.6 The whole hierarchy on one page (exam, 8 p)

Here's the story of Parts I and II as a staircase. Each class has a machine and a language that proves it's a real step up:

EXAM: FIVE CLASSES, EACH WITH A DEVICE AND AN EXAMPLE NOT IN THE SMALLER CLASS

Class	Machine / description	Example that needs this class
1. Regular	DFA, NFA, regular expression	$\{w \mid w \text{ has even length}\}$
2. Deterministic context-free	DPDA	$\{a^n b^n \mid n \geq 0\}$
3. Context-free	NPDA, context-free grammar	$\{ww^R \mid w \in \{a, b\}^*\}$
4. Recursive (decidable)	TM that always halts	$\{a^n b^n c^n \mid n \geq 0\}$
5. Recursively enumerable	TM (may run forever on "no")	the halting problem $\{\langle M \rangle w \mid M \text{ halts on } w\}$

The grading rewards **consistency**: the machine and the example must fit the class in the same row.

Where we're going next. A machine that can compute anything... surely it can answer any question? No. And the proof that it can't is one of the most beautiful arguments in all of mathematics.

13. What no machine can do FCS

You'd think a machine powerful enough to run any program could answer any well-defined yes/no question. This chapter shows that's false — and not for some exotic question, but for a very practical one: “**will this program ever stop?**”

13.1 Two kinds of “solving”

A Turing machine that answers questions can behave in two ways, and the difference is everything:

- A **decider** always halts: “yes” or “no”, every time. A language with a decider is **recursive** (= **decidable**).
- A **recognizer** halts with “yes” on every yes-instance, but on a no-instance it may say “no” — or **run forever**. A language with a recognizer is **recursively enumerable (RE)**.

WHY THE DIFFERENCE MATTERS

If a recognizer has been running for an hour, you don't know whether the answer is “no” or “not yet”. A decider never leaves you hanging.

One neat consequence: if a language **and** its complement are both RE, the language is decidable. Run both recognizers side by side, one step each; one of them must eventually say yes — and then you know the answer.

TRUE/FALSE TRAPS

“It can be correct to say that a Turing machine M is decidable” — **false**: *languages or problems* are decidable; a machine is a decider or not. “The set of (encoded) Turing machines is RE” — **true**: a machine can check whether a string is a valid encoding.

13.2 There are more problems than programs

Before the halting problem, a simpler shock. Machines are finite descriptions — strings — so we can list them all: machine 1, machine 2, machine 3... (**countably many**). But how many languages are there? Let's try to list them too and see what goes wrong:

CANTOR'S DIAGONAL TRICK

Suppose someone claims they've listed every language: $L_1, L_2, L_3, \dots$. List all strings too: $w_1, w_2, w_3, \dots$. Now build a new language D by going down the diagonal and doing the **opposite**: put w_i in D exactly when w_i is **not** in L_i .

Is D on the list? It can't be L_1 — they disagree about w_1 . It can't be L_2 — they disagree about w_2 . It can't be any L_i . So the list was incomplete. No list of languages can ever be complete: there are **uncountably many**.

So there are more languages than machines — most languages have **no machine at all**. That's abstract, though. Let's find a real one.

13.3 The halting problem

Question: given a machine M and an input w , does M halt on w ?

It is **RE**: just run M on w (with the universal machine), and say yes if it halts. But is it **decidable**? Suppose it were — suppose there is a perfect machine H that always correctly answers “does M halt on w ?”. Watch what we can build with it:

THE PROOF — A MACHINE THAT CONTRADICTS ITSELF

Build a troublemaker D . On input $\langle M \rangle$ (a machine’s description), D asks H : “does M halt when given its own description?” Then D does the **opposite**: if H says “halts”, D loops forever; if H says “loops”, D halts.

Now feed D its own description. Does D halt on $\langle D \rangle$?

- If it halts, then H said “ D loops” — so H was wrong.
- If it loops, then H said “ D halts” — wrong again.

Either way H is wrong. So a perfect H cannot exist: **the halting problem is undecidable**. (And since it is RE but not decidable, its complement is not even RE.)

Notice it’s the diagonal trick again: D is built to disagree with every machine on that machine’s own description.

13.4 Reductions: spreading the impossibility

Once one problem is known to be impossible, we can show others are impossible **without starting from scratch**. The argument goes: “if I could solve *your* problem, I could use it to solve the halting problem — and that’s impossible, so I can’t solve yours either.”

RECIPE: PROVING PROBLEM B UNDECIDABLE

1. Assume B has a decider.
2. Take any halting question “does M halt on w ?”. Show how to **transform** it (by an algorithm that always finishes) into a B-question with the **same answer**.
3. Then B’s decider would decide halting. Contradiction.

Direction matters: you transform the **known impossible** problem **into** the new one.

Example: “does M ever enter state q on input w ?” Given M, w , build M' : a copy of M with one extra state q_{new} , and wherever M would halt, M' instead goes to q_{new} . Then M' enters $q_{new} \Leftrightarrow M$ halts. So a solver for state-entry would solve halting — impossible.

Example: “does M halt when started on a blank tape?” Given M, w , build M_w : it first writes w on the tape, then behaves like M . M_w halts on blank $\Leftrightarrow M$ halts on w .

In fact almost every interesting question about what programs do — “is its language empty?”, “finite?”, “regular?” — is undecidable (**Rice’s theorem**). This is why no compiler can perfectly detect all infinite loops.

Where we’re going next. Among the problems we *can* solve, some take a blink and some would take longer than the age of the universe. Measuring that difference is complexity theory.

14. Fast and slow: P, NP and NP-completeness FCS

Think about a sudoku. **Solving** a hard one can take you an hour of trial and error. But if a friend hands you a filled-in grid and says “I solved it”, **checking** takes a minute: every row, column and box has 1–9. Checking is easy; finding seems hard.

That gap is the heart of this chapter. We’re no longer asking *can* a machine solve it (Chapter 13) — we’re asking *how fast*. And the rough line between “fast” and “hopeless” is: does the time grow like a **polynomial** (n^2, n^3) or like an **exponential** (2^n)? With $n = 100$, n^3 is a million steps — nothing. 2^{100} is more steps than there are atoms in... well, you’ll be waiting a while.

THE TWO CLASSES IN ONE SENTENCE EACH

P: problems you can **solve** in polynomial time. **NP**: problems where a proposed solution can be **checked** in polynomial time — like the sudoku. Every P problem is in NP (if you can solve it, you can check an answer by solving it). Whether $P = NP$ — whether every easy-to-check problem is also easy-to-solve — is the most famous open question in computer science.

14.1 Definitions

DEFINITIONS

- $f(n) = O(g(n))$ if there are c, n_0 with $f(n) \leq c g(n)$ for all $n \geq n_0$.
- A TM runs in time $T(n)$ if it halts within $T(n)$ steps on every input of length n .
- **P** = languages decided by a **deterministic** TM in polynomial time $O(n^k)$.
- **NP** = languages decided by a **nondeterministic** TM in polynomial time (every branch halts in $O(n^k)$, accept if some branch accepts). Equivalently: languages with a polynomial-time **verifier**: $w \in L$ iff there is a **certificate** c with $|c|$ polynomial in $|w|$ such that a deterministic poly-time machine accepts (w, c) .
- $P \subseteq NP$ (a deterministic machine is a special NTM). Simulating an NTM deterministically takes exponential time (Ch. 12), which is why we don’t know whether they are equal.
- Poly-time reduction $A \leq_p B$: a poly-time computable f with $w \in A \iff f(w) \in B$.
- B is **NP-hard** if $A \leq_p B$ for every $A \in NP$; **NP-complete** if also $B \in NP$.

COOK-LEVIN

SAT (is a propositional formula satisfiable?) is NP-complete. Then 3SAT, CLIQUE, VERTEX COVER, HAMILTONIAN PATH, SUBSET-SUM... are NP-complete by chains of reductions ($SAT \leq_p 3SAT \leq_p CLIQUE$).

Consequence: if any NP-complete problem is in P, then $P = NP$. So “If $P=NP$ then there is a polynomial algorithm for SAT” is **True** (FCS Q1).

RECIPE: PROVING PROBLEM B IS NP-COMPLETE

1. **B ∈ NP**: say what the certificate is and why checking it takes polynomial time.
2. **B is NP-hard**: pick a known NP-complete A (SAT, 3SAT, CLIQUE...) and give a reduction f from **A to B**.
3. Show f runs in polynomial time (output size polynomial).
4. Show $w \in A \Rightarrow f(w) \in B$ **and** $f(w) \in B \Rightarrow w \in A$.

NP-COMPLETE = “THE HARDEST PROBLEMS IN NP”

A reduction $A \leq_p B$ says “if I can solve B fast, I can solve A fast” — B is at least as hard as A. An NP-complete problem is one that every NP problem reduces to. So if **any single** NP-complete problem had a fast algorithm, **every** NP problem would, and $P = NP$. That’s why proving a problem NP-complete is taken as strong evidence that it has no fast algorithm.

14.2 SAT → 3SAT → CLIQUE, briefly

SAT → 3SAT: convert to CNF with new variables (to avoid blow-up), then split long clauses: $(l_1 \vee l_2 \vee l_3 \vee l_4) \rightarrow (l_1 \vee l_2 \vee y) \wedge (\neg y \vee l_3 \vee l_4)$.

3SAT → CLIQUE: for a formula with k clauses, make a vertex per literal occurrence; connect two vertices if they are in different clauses and are not contradictory (x vs $\neg x$). The formula is satisfiable iff the graph has a clique of size k (pick one true literal per clause).

14.3 A ☆ exam question: ◇-EQUATION is NP-complete

◇ is NAND: $0 \diamond 0 = 0 \diamond 1 = 1 \diamond 0 = 1, 1 \diamond 1 = 0$.

MODEL ANSWER

In NP. Certificate: an assignment of 0/1 to the variables (size $\leq$ input). Check: evaluate both sides bottom-up, one operation per ◇ — linear time — and compare.

NP-hard: reduce SAT. Given a formula φ over $\neg, \wedge, \vee$, translate it recursively: $T(x) = x, T(\neg\alpha) = T(\alpha) \diamond 1, T(\alpha \wedge \beta) = (T(\alpha) \diamond T(\beta)) \diamond 1, T(\alpha \vee \beta) = (T(\alpha) \diamond 1) \diamond (T(\beta) \diamond 1)$. These are correct since $x \diamond 1 = \neg x, \neg(a \diamond b) = a \wedge b$, and $(\neg a) \diamond (\neg b) = a \vee b$. Each subformula is used **once**, so $|T(\varphi)| \leq 5|\varphi|$: polynomial time. Output the equation $T(\varphi) = 1$.

For every assignment, $T(\varphi)$ has the same value as φ . So φ is satisfiable $\Leftrightarrow$ some assignment makes $T(\varphi) = 1 \Leftrightarrow$ the equation has a solution. Hence $\text{SAT} \leq_p \text{◇-EQUATION}$, which is therefore NP-complete.

14.4 A ☆ exam question: NP is closed under $\cup$ and $\cap$

MODEL ANSWER

Definition: NP is the class of languages L decided by a nondeterministic TM in polynomial time (as above).

Let $L_1, L_2 \in \text{NP}$, decided by NTMs N_1, N_2 in times $p_1(n), p_2(n)$.

Union: NTM N on w : nondeterministically choose $i \in \{1, 2\}$, run N_i on w . Some branch accepts $\Leftrightarrow N_1$ or N_2 has an accepting branch $\Leftrightarrow w \in L_1 \cup L_2$. Time $O(p_1(n) + p_2(n))$, polynomial.

Intersection: NTM N on w : run N_1 on w (nondeterministically); if that branch accepts, run N_2 on w (on a fresh copy of w); accept if it accepts. A branch accepts $\Leftrightarrow$ it contains an accepting run of N_1 followed by one of $N_2 \Leftrightarrow w \in L_1 \cap L_2$. Every branch takes at most $p_1(n) + p_2(n) + O(n)$ steps (copying w), polynomial. (Verifier version: the certificate for $\cap$ is the pair (c_1, c_2) ; for $\cup$ it is (i, c_i) .)

Part III

Haskell from zero

PL — 20 points on every exam

15. Haskell basics: types, functions, lists PL

Everything so far has been about machines that change state step by step — sticky notes, stacks, tapes, memory boxes. Now we meet a way of programming that throws that idea away completely. In Haskell nothing ever changes. There are no loops and no variables you can update. It sounds impossible to program like that — and the first few hours it feels strange — but it turns out to be a clean and powerful way to think, and it's 20 points on your exam. Let's take it gently, from zero.

THE MINDSET

In Java you write **instructions**: “set x to 0, loop, add...”. In Haskell you write **definitions**: “the sum of an empty list is 0; the sum of `x:xs` is `x` plus the sum of `xs`”. A function is like a maths function: same input $\Rightarrow$ always the same output, and it changes nothing else (no side effects). There are no loops — recursion does that job. Nothing is ever modified — you build new values.

15.1 Using GHCi

Run `ghci`, load a file with `:l file.hs`, reload with `:r`, ask a type with `:t`. Every `ghci>` line in this guide is real output.

```
ghci> 2 + 3 * 4
14
ghci> 10 `div` 3
3
ghci> 10 `mod` 3
1
ghci> 7 / 2
3.5
ghci> not (3 > 2) || True
True
ghci> "abc" ++ "def"
"abcdef"
ghci> 'h' : "ello"
"hello"
ghci> head [5,6,7]
5
ghci> tail [5,6,7]
[6,7]
ghci> length [5,6,7]
3
ghci> [1..5]
[1,2,3,4,5]
ghci> [x * 2 | x <- [1..5], odd x]
[2,6,10]
```

FUNCTION APPLICATION BINDS TIGHTEST

`f x + 1` means `(f x) + 1`. `f x y` is `f` applied to two arguments — no commas, no parentheses. To pass an expression, wrap it: `double (3 + 4)`, not `double 3 + 4`.

15.2 Types

Type	Values
------	--------

Int, Integer	whole numbers (fixed size / unlimited)
Double	decimals
Bool	True, False
Char	'a' (single quotes)
String	"abc" — it is just [Char]
[a]	list: all elements the same type
(a, b)	tuple: fixed size, types may differ
a -> b	function from a to b

Lowercase type names like `a` are **type variables**: “any type”. `length :: [a] -> Int` works for lists of anything (parametric polymorphism). A **constraint** like `Eq a =>` means “any type that supports `==`”; `Ord a` supports `<`; `Num a` supports `+`; `Show a` can be printed.

```
ghci> :t 'a'
'a' :: Char
ghci> :t "hi"
"hi" :: String
ghci> :t (True, 'x')
(True, 'x') :: (Bool, Char)
ghci> :t length
length :: Foldable t => t a -> Int
ghci> :t (==)
(==) :: Eq a => a -> a -> Bool
```

Note: modern GHC shows `length :: Foldable t => t a -> Int` because it works on more containers than lists. For lists, read `t a` as `[a]`: `length :: [a] -> Int`. On the exam, the list version is what is expected. The same applies to `sum`, `elem`, `foldr`, `foldl`.

15.3 Defining functions

```
double :: Int -> Int
double x = x * 2

add :: Int -> Int -> Int
add x y = x + y

isTeen :: Int -> Bool
isTeen age = age >= 13 && age <= 19
```

Read `add :: Int -> Int -> Int` as “takes an Int, then an Int, returns an Int”.

```
ghci> double 21
42
ghci> add 3 4
7
ghci> isTeen 15
True
```

Guards: many-way if

```
grade :: Int -> Char
grade points
  | points >= 90 = 'A'
  | points >= 70 = 'B'
  | points >= 50 = 'C'
  | otherwise   = 'F'
```

```
ghci> grade 75
'B'
ghci> grade 12
'F'
```

Pattern matching: define by cases on the shape of the input

```
describe :: [Int] -> String
describe []      = "empty"
describe [x]     = "one element: " ++ show x
describe (x:y:_) = "starts with " ++ show x ++ " and " ++ show y
```

Patterns are tried **top to bottom**. `[x]` = exactly one element; `(x:y:_)` = at least two; `_` = “anything, I don’t care”.

```
ghci> describe []
"empty"
ghci> describe [7]
"one element: 7"
ghci> describe [7,8,9]
"starts with 7 and 8"
```

where, let, case

```
bmi :: Double -> Double -> String
bmi weight height
  | v < 18.5 = "under"
  | v < 25.0 = "normal"
  | otherwise = "over"
  where
    v = weight / (height * height)

cylinder :: Double -> Double -> Double
cylinder r h =
  let side = 2 * pi * r * h
      top  = pi * r * r
  in side + 2 * top
```

```
sign :: Int -> String
sign n = case compare n 0 of
  LT -> "negative"
  EQ -> "zero"
  GT -> "positive"
```

```
ghci> bmi 70 1.8
"normal"
ghci> sign (-3)
"negative"
```

Tuples

```
swapPair :: (a, b) -> (b, a)
swapPair (x, y) = (y, x)

minMax :: [Int] -> (Int, Int)
minMax xs = (minimum xs, maximum xs)
```

```
ghci> swapPair (1, "one")
("one",1)
ghci> minMax [3,1,4,1,5]
(1,5)
ghci> fst (1, 'a')
1
ghci> snd (1, 'a')
'a'
```

15.4 Lists: the one data structure you must master

HOW LISTS ARE BUILT

A list is either **empty** `[]` or an element **in front of** a list: `x : xs`. `[1,2,3]` is only shorthand for `1 : (2 : (3 : []))`. That is why every list function has two cases: `[]` and `(x:xs)`.

Function	Does	Cost
<code>x : xs</code>	put in front	$O(1)$
<code>xs ++ ys</code>	join	$O(\text{length } xs)$
<code>head</code> , <code>tail</code>	first / all but first (crash on <code>[]</code>)	$O(1)$
<code>length</code> , <code>last</code> , <code>xs !! n</code>	walk the list	$O(n)$
<code>take n</code> , <code>drop n</code> , <code>splitAt n</code>	first n / without first n / both	$O(n)$
<code>reverse</code> , <code>elem</code> , <code>sum</code> , <code>maximum</code>	as named	$O(n)$
<code>replicate n x</code>	n copies	$O(n)$
<code>zip</code> , <code>unzip</code>	pairs up / splits pairs	$O(n)$
<code>words</code> , <code>unwords</code> , <code>lines</code>	split/join text	$O(n)$

```
ghci> take 2 [1,2,3,4]
[1,2]
ghci> drop 2 [1,2,3,4]
[3,4]
ghci> splitAt 2 "hello"
("he","llo")
ghci> replicate 3 'z'
"zzz"
ghci> zip [1,2,3] "abc"
[(1,'a'),(2,'b'),(3,'c')]
ghci> words "the cat sat"
["the","cat","sat"]
ghci> [1,2,3] !! 0
1
ghci> null []
True
```

16. Recursion and accumulators PL

No loops. So how do you do something for every element of a list? This chapter answers that, and it's the most important chapter of Part III: once recursion “clicks”, every Haskell exam question becomes a variation on two patterns you'll learn here.

HOW TO THINK RECURSIVELY (“TRUST THE RECURSION”)

To write `mySum (x:xs)`, **pretend** `mySum xs` **already works**. If `mySum [2,3]` is 5, then `mySum [1,2,3]` is `1 + 5`. You only have to answer two questions: **what's the answer for the empty list**, and **how do I combine the first element with the answer for the rest**.

RECIPE (USE IT FOR EVERY EXAM FUNCTION)

1. Write the **type signature**.
2. Write the **base case** (`f [] = ...`, or `f 0 = ...`).
3. Write the **recursive case** using `f xs` (smaller input).
4. **Trace** it on a 2–3 element list by hand.

16.1 Pattern 1 — plain recursion

```
mySum :: [Int] -> Int
mySum [] = 0
mySum (x:xs) = x + mySum xs

myProduct :: [Int] -> Int
myProduct [] = 1
myProduct (x:xs) = x * myProduct xs

myElem :: Eq a => a -> [a] -> Bool
myElem _ [] = False
myElem y (x:xs) = x == y || myElem y xs

myTake :: Int -> [a] -> [a]
myTake 0 _ = []
myTake _ [] = []
myTake n (x:xs) = x : myTake (n - 1) xs

myDrop :: Int -> [a] -> [a]
myDrop 0 xs = xs
myDrop _ [] = []
myDrop n (_:xs) = myDrop (n - 1) xs

factorial :: Integer -> Integer
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

```

ghci> mySum [1,2,3,4]
10
ghci> myProduct [1,2,3,4]
24
ghci> myElem 3 [1,2,3]
True
ghci> myTake 2 "hello"
"he"
ghci> myDrop 2 "hello"
"llo"
ghci> factorial 20
2432902008176640000

```

Trace $\text{mySum } [1,2,3] = 1 + \text{mySum } [2,3] = 1 + (2 + \text{mySum } [3]) = 1 + (2 + (3 + \text{mySum } [])) = 1 + (2 + (3 + 0)) = 6$. Notice the additions wait until the end — the computer must remember them on the stack.

Sorting — classic exam material

```

insertSorted :: Int -> [Int] -> [Int]
insertSorted x [] = [x]
insertSorted x (y:ys)
  | x <= y    = x : y : ys
  | otherwise = y : insertSorted x ys

insertionSort :: [Int] -> [Int]
insertionSort [] = []
insertionSort (x:xs) = insertSorted x (insertionSort xs)

quicksort :: [Int] -> [Int]
quicksort [] = []
quicksort (p:xs) = quicksort [x | x <- xs, x < p] ++ [p] ++ quicksort [x | x <- xs, x >= p]

merge :: [Int] -> [Int] -> [Int]
merge [] ys = ys
merge xs [] = xs
merge (x:xs) (y:ys)
  | x <= y    = x : merge xs (y:ys)
  | otherwise = y : merge (x:xs) ys

mergeSort :: [Int] -> [Int]
mergeSort [] = []
mergeSort [x] = [x]
mergeSort xs = merge (mergeSort l) (mergeSort r)
  where (l, r) = splitAt (length xs `div` 2) xs

```

```

ghci> insertionSort [5,2,9,1,5,6]
[1,2,5,5,6,9]
ghci> quicksort [5,2,9,1,5,6]
[1,2,5,5,6,9]
ghci> mergeSort [5,2,9,1,5,6]
[1,2,5,5,6,9]

```

16.2 Pattern 2 — accumulators and tail recursion

INTUITION

Carry the “answer so far” as an extra argument, like a loop variable. When the list is empty, the accumulator **is** the answer. The recursive call is the **last** thing the function does — that is **tail recursion**: nothing waits on the stack, so it runs like a loop in constant stack space.

Template

```
template :: [Int] -> Int
template xs = go xs 0
  where
    go []      acc = acc
    go (y:ys) acc = go ys (acc + y)
```

myRev in O(n) — asked on both 2023 exams (10 p)

```
myRev :: [a] -> [a]
myRev xs = go xs []
  where
    go []      acc = acc
    go (y:ys) acc = go ys (y : acc)
```

```
ghci> myRev "Hello"
"olleH"
ghci> myRev [1,2,3]
[3,2,1]
```

Trace: `go [1,2,3] []` → `go [2,3] [1]` → `go [3] [2,1]` → `go [] [3,2,1]`. One `:` per element = $O(n)$.
The accumulator works like a stack of plates: the first element ends up at the bottom.

NOT O(N)

```
slowRev :: [a] -> [a]
slowRev []      = []
slowRev (x:xs) = slowRev xs ++ [x]      -- correct, but  $O(n^2)$ !
```

`++` copies its left list each time: $1 + 2 + \dots + n = O(n^2)$.

Tail-recursive Fibonacci (2017 exam)

```
fibs :: Int -> [Int]
fibs n = go n 0 1
  where
    go 0 _ _ = []
    go k a b = a : go (k - 1) b (a + b)

fibTail :: Int -> Int      -- the n-th Fibonacci number, tail recursive
fibTail n = go n 0 1
  where
    go 0 a _ = a
    go k a b = go (k - 1) b (a + b)
```

```
ghci> fibs 10
[0,1,1,2,3,5,8,13,21,34]
ghci> fibTail 30
832040
```

Naive `fib n = fib (n-1) + fib (n-2)` is correct but takes exponential time (recomputes everything) and is not tail recursive.

Digits and numbers

```
listToInt :: [Int] -> Int
listToInt xs = go xs 0
  where
    go []      acc = acc
    go (d:ds) acc = go ds (acc * 10 + d)
```

```
intToList :: Int -> [Int]
intToList n
  | n < 10    = [n]
  | otherwise = intToList (n `div` 10) ++ [n `mod` 10]
```

```
ghci> listToInt [1,2,3]
123
ghci> intToList 4071
[4,0,7,1]
```

Run-length: compress and decompress (2023 exam, your last exam)

```
compress2 :: String -> [(Char, Int)]
compress2 []      = []
compress2 (c:cs) = go c 1 cs
  where
    go cur n []      = [(cur, n)]
    go cur n (x:xs)
      | x == cur     = go cur (n + 1) xs
      | otherwise    = (cur, n) : go x 1 xs
```

Alternative with `takeWhile`/`dropWhile`

```
compress :: String -> [(Char, Int)]
compress []      = []
compress (c:cs) = (c, 1 + length same) : compress rest
  where
    same = takeWhile (== c) cs
    rest = dropWhile (== c) cs
```

```
decompress :: [(Char, Int)] -> String
decompress []      = []
decompress ((c, n) : ps) = replicate n c ++ decompress ps
```

Without library functions

```
decompress2 :: [(Char, Int)] -> String
decompress2 []      = []
decompress2 ((c, 0) : ps) = decompress2 ps
decompress2 ((c, n) : ps) = c : decompress2 ((c, n - 1) : ps)
```

```
ghci> compress2 "aaaabbcccb"  
[('a',4),('b',2),('c',3),('b',1)]  
ghci> decompress [('a',4),('b',2),('c',3),('b',1)]  
"aaaabbcccb"
```

17. Higher-order functions: map, filter, fold, lambda

PL

By now you may have noticed you keep writing the same shape of recursion: walk down a list, do something to each element, or keep some elements, or combine them all. Programmers hate repeating themselves — so functional languages capture those shapes **once**, as functions that take *other functions* as arguments. This is where Haskell starts to feel powerful.

FUNCTIONS ARE VALUES

In Haskell a function can be **passed as an argument** and **returned as a result** (“first-class citizens”). A function that takes or returns a function is **higher-order**. `map`, `filter` and `fold` capture the three most common loops once and for all: **transform every element, keep some elements, combine all elements into one value**.

17.1 Lambdas, currying, sections, composition

```
applyTwice :: (a -> a) -> a -> a
applyTwice f x = f (f x)
```

```
ghci> applyTwice (\x -> x * 3) 2
18
ghci> applyTwice (++ "!") "hi"
"hi!!"
ghci> (\x y -> x * y + 1) 3 4
13
```

- **Lambda:** `\x -> x * 3` is a function without a name.
- **Currying:** every function takes one argument at a time. `add3 :: Int -> Int -> Int -> Int` really means `Int -> (Int -> (Int -> Int))`. So `add3 1` is a function waiting for two more (**partial application**).
- **Sections:** `(+ 5)`, `(* 2)`, `(> 3)`, `(`div` 2)` are operators with one side filled in.
- **Composition:** `(f . g) x = f (g x)` — read right to left. `f $ x = f x` (saves parentheses).

```
add3 :: Int -> Int -> Int -> Int
add3 x y z = x + y + z

addFive :: Int -> Int
addFive = (+ 5)
```

```
ghci> :t add3 1
add3 1 :: Int -> Int -> Int
ghci> (add3 1 2) 10
13
ghci> map (add3 1 2) [10, 20]
[13,23]
ghci> addFive 1
6
ghci> filter (> 3) [1,5,2,8]
[5,8]
```

```
countLongWords :: String -> Int
countLongWords = length . filter (\w -> length w > 3) . words
```

```
ghci> countLongWords "the quick brown fox jumps"
3
```

17.2 map and filter — write them yourself

```
map :: (a -> b) -> [a] -> [b]
map f [] = []
map f (x:xs) = f x : map f xs
```

```
filter :: (a -> Bool) -> [a] -> [a]
filter p [] = []
filter p (x:xs)
  | p x      = x : filter p xs
  | otherwise = filter p xs
```

```
myZipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
myZipWith f (x:xs) (y:ys) = f x y : myZipWith f xs ys
myZipWith _ _ = []
```

```
myTakeWhile :: (a -> Bool) -> [a] -> [a]
myTakeWhile _ [] = []
myTakeWhile p (x:xs)
  | p x      = x : myTakeWhile p xs
  | otherwise = []
```

```
ghci> myZipWith (+) [1,2,3] [10,20,30]
[11,22,33]
ghci> myTakeWhile (< 4) [1,2,3,7,1]
[1,2,3]
ghci> dropWhile (< 4) [1,2,3,7,1]
[7,1]
```

17.3 Folds

INTUITION

`foldl f z [x1,x2,x3] = f (f (f z x1) x2) x3` — start with `z`, eat the list from the **left**, accumulator first.

`foldr f z [x1,x2,x3] = f x1 (f x2 (f x3 z))` — replace each `:` by `f` and `[]` by `z`, element first.

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f acc [] = acc
foldl f acc (x:xs) = foldl f (f acc x) xs
```

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f acc [] = acc
foldr f acc (x:xs) = f x (foldr f acc xs)
```

```
ghci> foldl (-) 0 [1,2,3]
-6
ghci> foldr (-) 0 [1,2,3]
2
ghci> foldr (:) [] [1,2,3]
[1,2,3]
ghci> foldl (\acc x -> x : acc) [] [1,2,3]
[3,2,1]
```

foldl (-) 0 [1,2,3]

$((0-1)-2)-3 = -6$

foldr (-) 0 [1,2,3]

$1-(2-(3-0)) = 2$

WHICH FOLD?

- Building a list in the same order (map-like): **foldr**.
- Running total / “reading left to right” (sum, count, listToInt, reverse): **foldl**.
- Works on infinite lists (lazily): only **foldr**.

Exam 2023-05-05 Problem 6 (10 p): explain and evaluate

```
foo x y = filter (\w -> x == w) y
bar x y = foldl (\w -> \q -> (if q /= x then w + 1 else w)) 0 y
```

ANSWER

foo x y = filter keeps the elements `w` of `y` for which `x == w`: it returns all occurrences of `x` in `y`. **bar x y** = foldl with accumulator `w` starting at 0; for each element `q` it adds 1 if `q /= x`: it counts the elements **different** from `x`. The string has 12 characters, 4 of them 'c' :

```
ghci> foo 'c' "aabcccaacbba"
"cccc"
ghci> bar 'c' "aabcccaacbba"
8
```

17.4 Pipelines — how Haskell programmers really write

```
sumOfSquaresOfEvens :: [Int] -> Int
sumOfSquaresOfEvens xs = sum (map (^ 2) (filter even xs))
```

```
ghci> sumOfSquaresOfEvens [1..10]
220
```

```
ghci> map (\(c, n) -> replicate n c) [('a',2),('b',3)]
["aa","bbb"]
ghci> concat ["ab", "cd"]
"abcd"
ghci> concatMap (\(c, n) -> replicate n c) [('a',2),('b',3)]
"aabbb"
ghci> any even [1,3,5]
False
ghci> all odd [1,3,5]
True
```

18. Data types, type classes, laziness + problem bank

PL

So far we've only used types Haskell gave us: numbers, characters, lists. Real programs need their own — shapes, trees, expressions, stacks. Haskell lets you invent a type in one line, and then work with it using the very same pattern matching and recursion you already know. By the end of this chapter you'll have written a tree and a tiny interpreter.

18.1 Algebraic data types

INTUITION

`data` defines a new type by listing its possible **shapes**. Each shape has a **constructor** (capitalised) that may carry values. You take values apart with pattern matching — exactly like lists (which are themselves just `data List a = Nil | Cons a (List a)`).

```
data Shape = Circle Double | Rect Double Double
    deriving (Show, Eq)

area :: Shape -> Double
area (Circle r) = pi * r * r
area (Rect w h) = w * h
```

```
ghci> area (Circle 1)
3.141592653589793
ghci> area (Rect 3 4)
12.0
ghci> map area [Circle 1, Rect 2 5]
[3.141592653589793,10.0]
```

Maybe — “a value, or nothing”

```
safeHead :: [a] -> Maybe a
safeHead [] = Nothing
safeHead (x:_) = Just x

safeDiv :: Int -> Int -> Maybe Int
safeDiv _ 0 = Nothing
safeDiv a b = Just (a `div` b)

lookupAge :: String -> [(String, Int)] -> Maybe Int
lookupAge _ [] = Nothing
lookupAge k ((name, age) : rest)
    | k == name = Just age
    | otherwise = lookupAge k rest
```

```
ghci> safeHead ([] :: [Int])
Nothing
ghci> safeHead [4,5]
Just 4
ghci> safeDiv 10 0
Nothing
ghci> lookupAge "bo" [("al", 30), ("bo", 25)]
Just 25
```

18.2 Recursive data types and type constructors

The syllabus names “recursive data types” and “type constructors”. `List` and `Tree` below are **type constructors**: they are not types by themselves but build a type from another (`Tree Int`, `Tree String`).

```
data List a = Nil | Cons a (List a)
  deriving Show
```

```
lenL :: List a -> Int
lenL Nil      = 0
lenL (Cons _ xs) = 1 + lenL xs
```

```
ghci> lenL (Cons 'a' (Cons 'b' Nil))
2
```

```
data Tree a = Leaf | Node (Tree a) a (Tree a)
  deriving Show
```

```
insert :: Ord a => a -> Tree a -> Tree a
insert x Leaf = Node Leaf x Leaf
insert x t@(Node l v r)
  | x < v      = Node (insert x l) v r
  | x > v      = Node l v (insert x r)
  | otherwise = t
```

```
member :: Ord a => a -> Tree a -> Bool
member _ Leaf = False
member x (Node l v r)
  | x < v      = member x l
  | x > v      = member x r
  | otherwise = True
```

```
inorder :: Tree a -> [a]
inorder Leaf = []
inorder (Node l v r) = inorder l ++ [v] ++ inorder r
```

```
fromList :: Ord a => [a] -> Tree a
fromList = foldr insert Leaf
```

```
size :: Tree a -> Int
size Leaf = 0
size (Node l _ r) = size l + 1 + size r
```

```
depth :: Tree a -> Int
depth Leaf = 0
depth (Node l _ r) = 1 + max (depth l) (depth r)
```

```
ghci> fromList [5,3,8,1]
Node Leaf 1 (Node (Node Leaf 3 (Node Leaf 5 Leaf)) 8 Leaf)
ghci> inorder (fromList [5,3,8,1,4])
[1,3,4,5,8]
ghci> member 4 (fromList [5,3,8,1,4])
True
ghci> size (fromList [5,3,8,1,4])
5
ghci> depth (fromList [5,3,8,1,4])
3
```

Recipe for trees = same as lists: one equation per constructor; recurse into sub-trees.

An expression tree and its evaluator (an interpreter in 6 lines)

```
data Expr = Num Int
          | Add Expr Expr
          | Mul Expr Expr
          | Neg Expr
  deriving Show

eval :: Expr -> Int
eval (Num n)    = n
eval (Add a b)  = eval a + eval b
eval (Mul a b)  = eval a * eval b
eval (Neg e)    = negate (eval e)
```

```
ghci> eval (Add (Num 2) (Mul (Num 3) (Num 4)))
14
```

18.3 Type classes (ad hoc polymorphism)

A type class is a set of types that support certain operations — like a Java interface. `Eq`, `Ord`, `Show`, `Num` are builtin; deriving (`Show`, `Eq`) generates instances automatically.

```
class Describable a where
  describe :: a -> String

instance Describable Bool where
  describe True  = "yes"
  describe False = "no"

instance Describable Shape where
  describe (Circle _) = "a round shape"
  describe (Rect _ _) = "a box"
```

```
ghci> describe True
"yes"
ghci> describe (Rect 1 2)
"a box"
```

18.4 An ADT in Haskell (information hiding via modules)

```
newtype Stack a = Stack [a]

empty :: Stack a
empty = Stack []

push :: a -> Stack a -> Stack a
push x (Stack xs) = Stack (x : xs)

pop :: Stack a -> Maybe (Stack a)
pop (Stack []) = Nothing
pop (Stack (_:xs)) = Just (Stack xs)

top :: Stack a -> Maybe a
top (Stack []) = Nothing
top (Stack (x:_)) = Just x

isEmpty :: Stack a -> Bool
isEmpty (Stack xs) = null xs
```

In a real module you export `module Stack (Stack, empty, push, pop, top, isEmpty) where` — the type `Stack` without its constructor. Clients can't see or build the list inside: that is **information hiding**.

```
ghci> top (push 3 (push 2 empty))
Just 3
ghci> isEmpty empty
True
```

18.5 Laziness

Haskell evaluates an expression only when its value is needed (**lazy / non-strict** evaluation, call-by-need). So infinite lists are fine as long as you use only a finite part.

```
naturals :: [Integer]
naturals = [0..]

squares :: [Integer]
squares = [n * n | n <- naturals]

fibsLazy :: [Integer]
fibsLazy = 0 : 1 : zipWith (+) fibsLazy (tail fibsLazy)
```

```
ghci> take 5 squares
[0,1,4,9,16]
ghci> 16 `elem` takeWhile (<= 16) squares
True
ghci> take 10 fibsLazy
[0,1,1,2,3,5,8,13,21,34]
```

18.6 Problem bank (verified answers)

Try each one yourself first. Then compare.

```
countOcc :: Eq a => a -> [a] -> Int
countOcc _ [] = 0
countOcc y (x:xs)
  | x == y    = 1 + countOcc y xs
  | otherwise = countOcc y xs
```

```
myLength :: [a] -> Int
myLength xs = go xs 0
  where
    go []      acc = acc
    go (_:ys) acc = go ys (acc + 1)
```

```
mapF :: (a -> b) -> [a] -> [b]
mapF f xs = foldr (\x acc -> f x : acc) [] xs

filterF :: (a -> Bool) -> [a] -> [a]
filterF p xs = foldr (\x acc -> if p x then x : acc else acc) [] xs
```

```
myLast :: [a] -> a
myLast [x] = x
myLast (_:xs) = myLast xs

myMaximum :: [Int] -> Int
myMaximum [x] = x
myMaximum (x:xs) = max x (myMaximum xs)

elemAt :: [a] -> Int -> a
elemAt (x:_) 0 = x
elemAt (_:xs) n = elemAt xs (n - 1)
```

```
isPal :: Eq a => [a] -> Bool
isPal xs = xs == rev xs []
  where
    rev []      acc = acc
    rev (y:ys) acc = rev ys (y : acc)
```

```
dedup :: Eq a => [a] -> [a]
dedup (x:y:rest)
  | x == y    = dedup (y : rest)
  | otherwise = x : dedup (y : rest)
dedup xs = xs           -- 0 or 1 element left
```

```
myZip :: [a] -> [b] -> [(a, b)]
myZip (x:xs) (y:ys) = (x, y) : myZip xs ys
myZip _      _      = []

myAppend :: [a] -> [a] -> [a]
myAppend []      ys = ys
myAppend (x:xs) ys = x : myAppend xs ys
```

```
ghci> countOcc 'a' "banana"
3
ghci> myLength "abcd"
4
ghci> mapF (*10) [1,2]
[10,20]
ghci> myMaximum [3,9,2]
9
ghci> isPal "racecar"
True
ghci> dedup [1,1,2,3,3,3,1]
[1,2,3,1]
ghci> myAppend "ab" "cd"
"abcd"
```

READING TYPE ERRORS

“Couldn’t match expected type `[a]` with actual type `a`” almost always means you wrote `xs ++ x` (element where a list was needed) — use `xs ++ [x]` or `x : xs`. “Non-exhaustive patterns” means you forgot a case (usually `[]`).

Part IV

How a program comes alive — ideas behind every language

PL

19. Names, scope and types PL

In Part I we saw how an interpreter turns text into a tree. Now the tree has to *run* — and the first thing it runs into is **names**. Your code says `x`. Which `x`? Where does its value live? When is it created, when does it die, and what is it allowed to hold? Every language has to answer these questions, and the answers are what the “explain” questions on your exam are about. Once you see them as *design decisions* — choices a language designer made, each with a price — they stop being a list of terms and start making sense.

THE SHAPE OF A FULL-POINTS “EXPLAIN” ANSWER

(1) a precise definition, (2) how it works, (3) a tiny example, (4) an advantage and a disadvantage, (5) how it differs from the neighbouring concept. Five sentences like that almost always get full points. We’ll use that shape throughout Part IV.

19.1 Four ways to think about programs

Before the details, the big picture. Languages come in families — **paradigms** — depending on what they think a program *is*:

Paradigm	A program is...	Examples	You think in...
Imperative	a list of commands that change the machine’s state	C, Pascal	variables, assignment, loops
Object-oriented	objects that hold state and send each other messages	Java, C++, Python, Smalltalk	classes, inheritance
Functional	definitions of functions, evaluated like maths	Haskell, ML, Lisp, Erlang	recursion, no changing values
Logic	facts and rules; you ask questions	Prolog	<i>what</i> is true, not <i>how</i> to compute

19.2 What a variable really is

In an imperative language, a variable is a **named box in memory**. It has six attributes, and it’s worth being able to list them: **name**, **address** (where the box is), **value** (what’s in it), **type** (what may go in it), **lifetime** (when the box exists) and **scope** (where in the code the name can be used).

A **binding** is any association between a name and one of these things, and **binding time** is *when* it happens: some are fixed when the language is designed (`*` means multiply), some at compile time (the type of `int x` in Java), and some only at run time (the value of `x`, or the address of a local variable). Binding that happens before the program runs and never changes is **static**; binding that happens or changes while it runs is **dynamic**. That one pair of words — static/dynamic — comes back again and again in this Part.

Where do the boxes live? A running program’s memory looks like this:

stack: one activation record per active call (return address, parameters, locals) — grows ↓
free memory
heap: objects made with new / malloc — grows ↑
global and static data, constants
program code

Memory of a running program

- **Static** variables (globals) exist for the whole run.
- **Stack-dynamic** variables (locals) are created when a function is called and destroyed when it returns. This is what makes recursion possible: each call gets its own fresh set.
- **Heap-dynamic** variables are created explicitly (`new`) and live until freed or garbage-collected.

19.3 Two meanings of “=” — the functional twist

Here’s a question that appeared on the exam: in both Java and Haskell you can write `a = b + 4`. Why do we call it **assignment** in one and **naming a value** in the other? Answer it with the box picture:

EXAM (6 P): VARIABLES IN IMPERATIVE VS FUNCTIONAL LANGUAGES

In an **imperative** language a variable is an abstraction of a **memory cell**. `a = b + 4` computes `b + 4` and **stores** it in the cell called `a`, overwriting what was there. Later the cell can be changed again (`a = a + 1`), so the meaning of `a` depends on *when* you look: the program has **state**, and assignment is a **side effect**.

In a **pure functional** language there are no cells to change. `a = b + 4` is a **definition**: it binds the name `a` to the value of `b + 4`, like “let $a = b + 4$ ” in mathematics, and that can never change within its scope. (`a = a + 1` would be a circular definition, not an update.) Because a name always means the same value, you can replace it by its value anywhere — **referential transparency** — which makes programs easier to reason about, test and run in parallel. Repetition is done by recursion rather than by updating loop variables. In short: **a changeable storage location** versus **a permanent name for a value**.

19.4 Scope: which `x` is it?

Suppose there are two variables called `x` — a global one, and a local one inside some function. A function that mentions `x` without declaring it: which one does it get? There are two possible rules.

- **Static (lexical) scope**: look at the **program text**. The `x` is the one declared in the nearest block that *surrounds the function where it is written*. You can answer by reading the code — no need to run it.
- **Dynamic scope**: look at the **calls at run time**. The `x` is the most recent one created by *whoever called you* (or their caller, and so on).

Here’s the same little program in two real languages. Python uses static scope, the bash shell uses dynamic scope — and they give different answers:

```
f sees x = 1
```

```
f sees x = 2
```

In Python, `f` is written at the top level, so its `x` is the global one: 1. In bash, `f` was *called by* `g`, which had just made its own `x = 2` — so `f` sees 2.

Almost every modern language uses static scope, and now you can see why: with dynamic scope, what a function does depends on who called it, so you can't understand a function by reading it. Static scope is readable and can be checked by the compiler. Dynamic scope is flexible but confusing.

Don't mix up **scope** (where in the text a name is visible) with **lifetime** (when the box exists in memory). A C `static` local variable has local scope but lives for the whole program.

19.5 Types: what may go in the box

A **type system** is the set of rules saying which values and operations go together — you can add two numbers, but what does adding a number to a list mean? **Type checking** enforces those rules. The big design choice is *when*:

	Static typing (Java, C, Haskell)	Dynamic typing (Python, JavaScript, Lisp)
When checked	at compile time, before running	at run time, when the operation happens
Good	errors found early; faster code; documents intent	flexible; less to write
Bad	more rigid; must satisfy the compiler	errors show up late, maybe in front of a user; slower

Other words you should know: **strong** typing means type errors are always caught (Haskell); **weak** means some slip through (C with casts). **Type inference** means the compiler works out the types itself — Haskell can do this for almost everything. **Coercion** is automatic conversion, like Java turning `1 + 2.5` into `1.0 + 2.5`.

A Python function `def example(a, b): return a + b` happily works on numbers, strings and lists. That's dynamic typing together with an overloaded `+` — sometimes called “duck typing”: if it can be added, add it.

Where we're going next. Names, boxes, types — the ingredients. Next: how a program *moves* — choosing, repeating, and calling functions, including the famous question of how arguments get passed.

20. Control flow and subprograms PL

20.1 The minimum you need to compute anything

Here's a surprising fact. An imperative language that has only assignment (`x = expression`) can't compute much. Add just two things — a way to **choose** (`if`) and a way to **jump** (`goto`) — and it can compute anything a Turing machine can. Everything else — `while`, `for`, `switch` — is there to make programs *readable*, because raw `goto` turns code into spaghetti.

- **Choosing:** `if-then-else` (use blocks `{ }` — otherwise nested ifs are ambiguous, the “dangling else”), `switch / case` (in C you need `break`, or execution falls through to the next case), and **pattern matching** in Haskell.
- **Repeating:** `while` checks first; `do ... while` runs the body at least once. `break` leaves the loop, `continue` skips to the next round. **Iterators** (`for fruit in basket`) loop over a collection without an index — simpler and harder to get wrong.
- **Guarded commands** are an unusual, elegant idea: you list several (condition $\rightarrow$ action) pairs, and **any** one whose condition is true may be chosen. A loop of them keeps going until no condition holds. Example: “while some neighbours are in the wrong order, swap them” — it sorts, whatever order you pick the swaps in. The point is to express only what matters for correctness.

20.2 Subprograms: abstraction for actions

A subprogram (function, procedure, method) lets you give a name to a piece of work and reuse it — so you think “sort the list”, not the twenty lines that do it. That's **process abstraction**, the partner of the data abstraction in Chapter 21. Small vocabulary: a **procedure** does something (changes state), a **function** returns a value, a **method** belongs to an object.

Every call pushes an **activation record** onto the stack — the return address, the parameters and the local variables — and pops it at return. That's why a recursive function can call itself a thousand times without the calls trampling each other: each has its own record.

Language designers must decide lots of things about subprograms: How are argument types checked? Can functions be nested inside functions? Can a function be passed as an argument (as in Haskell's `map`)? Can two functions share a name if their parameter types differ (**overloading**)? Can one definition work for many types (**generics**)? May functions have side effects? How many values can they return?

Flexible parameters

Python shows several parameter features in one place — keyword arguments, default values, and a variable number of arguments:

```
20 12
1  ()
1  (2, 3, 4, 5)
20 12 {'myname': 'Halvard'}
```

Coroutines: two functions taking turns

Normally a function call is master and servant: the caller waits until the callee is completely finished. A **coroutine** is different — two routines pass control back and forth, each **resuming where it left off**. Python generators are exactly this:

```
producer: made a
consumer: used a
producer: made b
consumer: used b
producer: made c
consumer: used c
```

The producer pauses at `yield`, the consumer uses the item, then the producer continues from where it stopped. It feels like two things running at once, on one processor.

20.3 Parameter passing: by value, by reference and the others

THE IDEA IN PLAIN WORDS

By value = you give the function a **photocopy**. It can scribble on the copy; your original is untouched. **By reference** = you give the function **the address of your original**. Whatever it writes, it writes on your original.

20.3.1 Vocabulary first

- **Formal parameter**: the name in the function *definition* (`x` in `void f(int x)`).
- **Actual parameter (argument)**: the expression in the *call* (`a+1` in `f(a+1)`).
- **Modes** (semantic models): **in** (data goes into the function), **out** (data comes back to the caller), **in-out** (both).
- **Implementation**: essentially either **copy the value** or **pass an access path (address)**.

20.3.2 The five methods

Method	Mode	How it works	Problems
By value	in	The value of the actual parameter is copied into the formal parameter, which acts as a local variable. Changes inside do not affect the caller.	Copying large data (arrays, structs) costs time and memory.
By result	out	No value is passed in. At return, the formal parameter's final value is copied back to the actual parameter (which must be a variable).	<code>procA(a, a)</code> : two results copied to the same variable — the order of copying decides the value. <code>procA(a[i], i)</code> : is the address of <code>a[i]</code> computed at the call or at the return?

By value-result (copy-in/copy-out)	in-out	Copied in at the call and copied back at the return.	Same aliasing/order problems as by result; copying cost twice. <code>procA(4+4)</code> : an expression cannot receive a result.
By reference	in-out	The address of the actual parameter is passed. The formal parameter is an alias of the caller's variable; every read and write goes to the original.	Aliasing (two names for one variable: <code>f(a, a)</code>); unwanted side effects "open up" the caller's data; indirect access is slightly slower.
By name	in-out	The argument expression is passed <i>unevaluated</i> and re-evaluated every time the parameter is used (like textual substitution; Algol 60).	Hard to understand and slow; basis for lazy evaluation (Haskell uses call-by-need).

20.3.3 Verified example

```
#include <iostream>
using namespace std;

void swapByValue(int x, int y)    { int t = x; x = y; y = t; }
void swapByReference(int &x, int &y) { int t = x; x = y; y = t; }

void fRef(int &x, int &y) { x = x + 1; y = y * 2; }           // called as fRef(a, a)

int main() {
    int a = 1, b = 2;
    swapByValue(a, b);
    cout << "after swapByValue(a, b):    a = " << a << ", b = " << b << "\n";
    swapByReference(a, b);
    cout << "after swapByReference(a, b): a = " << a << ", b = " << b << "\n";

    int c = 5;
    fRef(c, c);                                           // x and y are ALIASES of c
    cout << "by reference, f(c, c) with c = 5:    c = " << c << "\n";

    // pass-by-value-result, simulated: copy in, work on copies, copy out (left to right)
    int d = 5;
    int x = d, y = d;           // copy in
    x = x + 1; y = y * 2;        // body works on local copies
    d = x; d = y;               // copy out: x first, then y (the last copy wins)
    cout << "by value-result, f(d, d) with d = 5: d = " << d << " (would be " << x
        << " if copied out in the other order)\n";
}
```

```
after swapByValue(a, b):    a = 1, b = 2
after swapByReference(a, b): a = 2, b = 1
by reference, f(c, c) with c = 5:    c = 12
by value-result, f(d, d) with d = 5: d = 10 (would be 6 if copied out in the other order)
```

The last two lines are the aliasing problem in numbers: with the **same** function body and the **same** call `f(c, c)`, pass-by-reference gives 12, pass-by-value-result gives 10 (or 6, depending on copy-back order).

MODEL ANSWER: “EXPLAIN THE DIFFERENCES BETWEEN PASS-BY-VALUE AND PASS-BY-REFERENCE. ADVANTAGES AND DISADVANTAGES?” (6 P)

Pass-by-value: at the call, the *value* of the actual parameter is evaluated and **copied** into the formal parameter, which then behaves like a local variable. Changes to it inside the subprogram do not affect the caller (in-mode). *Advantages:* safe — no side effects on the caller’s variables, easy to reason about; the argument can be any expression ($f(a+1)$); access to the parameter is fast (local). *Disadvantages:* copying large objects costs time and memory; it cannot be used to return results through parameters.

Pass-by-reference: the *address* of the actual parameter is passed, so the formal parameter is an **alias** for the caller’s variable; all reads and writes go directly to the original (in-out mode). *Advantages:* efficient — no copying, only an address is passed, even for large structures; the subprogram can return results through parameters (e.g. `swap(a, b)`). *Disadvantages:* side effects — the caller’s data can be changed unintentionally; **aliasing** (e.g. `f(a, a)` , or an array element and the whole array) makes programs hard to understand and verify; each access goes through an indirection (slightly slower); the argument must be a variable, not an arbitrary expression.

Example: `swap(a, b)` works only with pass-by-reference; with pass-by-value it swaps the copies and `a` , `b` are unchanged. (Java passes everything by value, but for objects the value *is* a reference — so the object can be modified, but the caller’s variable cannot be made to point elsewhere.)

21. Abstraction, ADTs and object-orientation PL

Programs get big. A real system has millions of lines, written by hundreds of people, and nobody can hold it all in their head. So how does anyone build anything? The same way you use a car without understanding its engine: through **abstraction**. You get a steering wheel and pedals — a small, clear interface — and everything behind it is somebody else's problem. This chapter is about the language features that let programmers build those “steering wheels” for their code, and it's where your exam asks you to *explain* things precisely. Precision is exactly what cost points last time, so we'll be careful with every word.

THE IDEA IN PLAIN WORDS

Think of a **medicine capsule**. **Encapsulation** is putting the ingredients *together in one capsule*: the data and the operations that work on it are packed into one unit (a class, a module, an ADT).

Information hiding is making the capsule *opaque*: from outside you can't see or touch what is inside; you can only use it the intended way (swallow it = call its public operations).

They are two different things. You can have a capsule you can see through: a Python class groups data and methods (encapsulation) but anyone can still write `s.data = []` (no real hiding).

21.1 The two definitions, precisely

ENCAPSULATION

An abstract data type is specified as **one syntactic unit** (class, module, package) that contains the **representation** of the data *together with* the **operations** on it. Other parts of the program can use the type **only through that definition**.

Advantages: better **organisation** (everything about stacks is in one place), code is **easier to read and understand**, **easier to modify and maintain**, and units can be **compiled separately**.

INFORMATION HIDING

The **internal representation** of the unit (its data fields, helper functions, implementation details) is **not accessible from outside** the unit. Clients see only the **interface** (the public operations and their types).

Advantages: **reliability** — code that is not “authorised” cannot read or change the internal data, so the unit can guarantee its own **invariants** (e.g. `top` always points at the top element). And **changeability** — the representation can be replaced (array → linked list) without changing any client code, because no client could depend on it. It also lowers what a programmer must know to use the unit.

How languages provide it: access modifiers `private` / `protected` / `public` (Java, C++, C#); separate definition and implementation modules (Modula-2: the definition module is public, the implementation module is hidden); export lists (Haskell modules exporting a type but not its constructor; ML `abstype`).

21.2 The same stack, showing both ideas (compiles and runs)

```
#include <iostream>

class Stack {           // ENCAPSULATION: data + operations in one unit
private:               // INFORMATION HIDING: invisible from outside
    int data[100];
    int top = -1;
public:                // the interface: the only way in
    void push(int x) { if (top < 99) data[++top] = x; }
    int pop()          { return data[top--]; }
    bool isEmpty()     { return top == -1; }
};

int main() {
    Stack s;
    s.push(1); s.push(2); s.push(3);
    while (!s.isEmpty()) std::cout << s.pop() << " ";
    std::cout << "\n";
#ifdef CHEAT
    s.top = 50;         // try to break the stack from outside
#endif
}
```

3 2 1

Now “cheat” by compiling with the line `s.top = 50;` switched on — the compiler refuses. *That* is information hiding: the language enforces it.

error: ‘int Stack::top’ is private within this context

MODEL ANSWER: “EXPLAIN ENCAPSULATION” (WRITE ALL FOUR PARTS)

(1) Definition. Encapsulation means that a data type’s representation and the operations on it are defined together as one syntactic unit (a class, module or package), so the rest of the program uses the type only through that unit’s definition. **(2) Purpose.** It is a way of achieving data abstraction: the user thinks in terms of “a stack with push/pop”, not in terms of arrays and indices. **(3) Example.** A class `Stack` containing the array and the `top` index together with `push`, `pop`, `isEmpty`. **(4) Advantages.** Better organisation of the code, easier to read, understand and modify, and supports separate compilation of the unit. It is often combined with, but is not the same as, information hiding.

MODEL ANSWER: “EXPLAIN INFORMATION HIDING” (WRITE ALL FIVE PARTS)

(1) Definition. Information hiding means that the internal representation and implementation details of a unit are not visible or accessible outside it; clients can only use the public interface. **(2) Mechanism.** E.g. private fields with public methods in Java/C++, an export list in a Haskell module that exports the type but not its constructors, or Modula-2's separate definition and implementation modules. **(3) Example.** In the stack class, data and top are private; `s.top = 50` from outside is a compile error, so nobody can put the stack into an inconsistent state. **(4) Advantages.** Reliability/integrity (invariants cannot be broken by outside code), the implementation can be changed without affecting clients (low coupling), and users need to know less. **(5) Relation to encapsulation.** Encapsulation *groups* data and operations; information hiding *restricts access* to the grouped internals. Together they give an abstract data type. (Possible disadvantage to mention: some overhead — calls to getters/setters — and sometimes less flexibility, e.g. a subclass or a trusted piece of code may legitimately need internal access; that is why languages add *protected* or “friend”.)

21.3 The neighbours — define these precisely too

Term	Precise definition	Tiny example
Abstraction	Representing something by its essential properties and ignoring details that are irrelevant at the current level. Two kinds: process abstraction (subprograms: you call <code>sort(list)</code> without knowing the algorithm) and data abstraction (abstract data types: you use a stack through <code>push/pop</code> without knowing its representation). Humans need it to manage complexity.	<code>sort(xs)</code> ; a Stack ADT
Abstract data type (ADT)	A data type whose values can only be created and used through a set of operations, defined by what the operations do (e.g. equations <code>pop(push(s,e)) = s</code> , <code>top(push(s,e)) = e</code>), not how. Implemented with encapsulation + information hiding.	stack: <code>empty</code> , <code>push</code> , <code>pop</code> , <code>top</code> , <code>isEmpty</code>
Inheritance	A class (subclass) is defined as an extension of an existing class (superclass): it inherits its fields and methods and can add new ones or override existing ones. Gives code reuse and an “is-a” hierarchy. Single (Java) or multiple (C++).	<code>class Dog extends Animal</code>
Polymorphism	“Many forms”: one name/interface can be used with values of different types . Subtype polymorphism: a variable of type <code>Animal</code> can hold a <code>Dog</code> , and <code>a.speak()</code> runs the <code>Dog</code> version (via dynamic binding). Parametric : one definition works for all types (generics; Haskell <code>length :: [a] -> Int</code>). Ad hoc : overloading — the same name has different implementations for different types (<code>+</code> on ints and floats; Haskell type classes).	<code>animal.speak()</code> ; <code>length</code> on any list

Dynamic binding	Which method implementation a call runs is decided at run time from the object's actual class (not from the variable's declared type). Flexible but costs a little speed; static binding decides at compile time (faster).	virtual methods in C++, all methods in Java
------------------------	---	---

WHY YOUR ANSWER WAS "TOO VAGUE"

"Encapsulation is hiding the data" is (a) vague and (b) mixes up the two concepts. Examiners want: a precise definition, the mechanism, an example, the advantages, and the difference from the related concept. The model answers above have exactly that structure — use it for *every* "explain the concept X" question.

21.4 Describing a type by what it does, not how

```

ADT stack is elem + boolean +
operations:  empty : -> stack          push : stack × elem -> stack
              pop  : stack -> stack      top  : stack -> elem          isEmpty : stack -> boolean
equations:   isEmpty(empty) = true      isEmpty(push(s,e)) = false
              pop(push(s,e)) = s         pop(empty) = error
              top(push(s,e)) = e         top(empty) = error

```

The equations say **what** the operations do, with no word about arrays or lists — that is data abstraction. A program must also contain implementation details; the ADT idea combines the high-level description with an implementation hidden inside a unit. The same stack can be written in Modula-2 (definition module = interface, implementation module = hidden), Java (`private` fields), ML (`abstype ... with ... end`) and Python (class, but nothing truly hidden).

21.5 Object-oriented programming: the big design questions

- **Core ideas:** data at the centre, encapsulation, inheritance, dynamic binding, modelling. Objects send and answer **messages**; classes vs instances; class methods (static) vs instance methods.
- **History:** Simula 67 (first classes), Smalltalk (1972, pure OO), C++ and Java (1980s–90s).
- **Language support:** pure OO (Smalltalk, Ruby: everything is an object, simple and elegant but slower); mostly OO (Java: everything except primitive types, which have wrapper classes; objects on the heap; single inheritance + interfaces; C#, Eiffel); OO added to an existing language (C++: mixed type system, multiple inheritance, access control; Objective-C, Ada 95, CLOS).
- **OO vs ADT:** ADTs are hard to **reuse** with small changes; inheritance lets a subclass reuse and modify. But access rules like `protected` let subclasses see internals — this **breaks strict information hiding**. Overriding can cause **shadowing**.
- **Inheritance: single or multiple?** Multiple is more expressive but complex (the **diamond problem**: two parents inherit the same method from a grandparent — which version?). Java: single inheritance + multiple interfaces.
- **Dynamic vs static binding:** dynamic = flexible (polymorphic calls), costs a lookup at run time; static = faster. C++ lets you choose (`virtual`); Java uses dynamic by default.
- **Type checking of overridden methods:** dynamic checking costs time; static checking requires the overriding method to have compatible parameters/return type — cheaper and safer.
- **Where are objects allocated?** Stack (fast, freed automatically, but lifetime tied to the block; C++ allows it) vs heap (flexible lifetime, needs GC or `delete` ; Java always).

- **Nested classes:** a class inside a class — hides a helper class, can access the outer class's members; but more complex.
- **Getters and setters** keep fields private while allowing controlled access; Python's `@property` makes them look like plain attributes.

22. Interpreters: lexer, parser, executor PL

This is the chapter where the whole story comes together. Everything from Part I — finite automata, regular expressions, grammars, stacks, parse trees — and everything from Part IV — names, scopes, environments, call stacks — are the parts of one machine: the thing that runs your code.

THE IDEA IN PLAIN WORDS

Your program is, to the computer, just a long **string of characters**: `x = 2 + 3 * 4; .` To run it, the language implementation works like reading a sentence in a foreign language:

1. **Lexer (scanner)** — find the *words*. Group characters into meaningful chunks and label them: `x` is an identifier, `=` is an assignment sign, `2` is a number... Throw away spaces and comments.
2. **Parser** — find the *grammar/structure* of the sentence. Check that the words are in a legal order and build a **tree** that shows what belongs to what: the `3 * 4` is done first and then added to `2`.
3. **Executor (interpreter)** — *do what the sentence says*. Walk the tree, compute the values, store `14` in `x`.



The pipeline (Crafting Interpreters calls it “the mountain”)

22.1 Lexer (scanner)

Input	the source code as a string / stream of characters
Output	a list (sequence) of tokens
Task	Group characters into lexemes (the smallest meaningful pieces: <code>while</code> , <code>x1</code> , <code>3.14</code> , <code>>=</code>) and classify each one as a token . A token typically stores: its type (IDENTIFIER, NUMBER, PLUS, keyword WHILE...), the lexeme text, the literal value (e.g. the number 3.14) and the line number (for error messages). It discards whitespace and comments, recognises keywords, and reports lexical errors (an illegal character like <code>#</code> , an unterminated string).
Theory	Tokens are described by regular expressions (<code>identifier = [a-zA-Z_][a-zA-Z0-9_]*</code>), so the lexer is a finite automaton . It follows the “longest match” rule: <code>>=</code> is one token, not <code>></code> then <code>=</code> ; <code>whilex</code> is an identifier, not the keyword <code>while</code> .
Data structures	a character buffer/string with position pointers (start of the current lexeme, current character); a hash table of keywords (reserved words); the output list/array of tokens ; sometimes it inserts identifiers in a symbol table .

22.2 Parser

Input	the list of tokens from the lexer
Output	a syntax tree — usually an abstract syntax tree (AST) — or syntax errors

Task	Check that the token sequence follows the grammar of the language (the syntax), and build a tree that shows the structure of the program: which operands belong to which operator, which statements are inside which loop/block/function. Precedence and associativity are decided here: from <code>2 + 3 * 4</code> the parser builds <code>Add(2, Mul(3, 4))</code>. It reports syntax errors with a position (“expected ‘)’ at line 7”) and tries to recover (skip to the next statement) to report more than one error.
Theory	The syntax is described by a context-free grammar (BNF), because programs have nesting (parentheses, blocks inside blocks) that regular expressions cannot describe. A parser is in effect a pushdown automaton: it needs a stack. Common techniques: recursive descent (top-down: one function per grammar rule; the call stack is the stack), LL, and LR/LALR (bottom-up, e.g. generated by yacc/bison). An ambiguous grammar is a problem for the parser, since one program would get two trees (two meanings).
Data structures	the token list with an index to the current token (lookahead); a stack (explicit in LR parsers, or the recursion call stack in recursive descent); the output tree (nodes = operators, statements; leaves = literals, identifiers); often a parse table (LL/LR).

22.3 Between parser and executor: static (semantic) analysis

Things the grammar cannot check: Is this variable declared? Which declaration does this name refer to (scope resolution)? Do the types fit (in statically typed languages)? This pass walks the tree and uses a **symbol table** (hash maps, one per scope, stacked/chained for nested scopes). It may annotate the tree with the results.

22.4 Executor (the interpreting part)

Input	the (annotated) syntax tree
Output	the program's behaviour: values, output, side effects; or runtime errors
Task	Walks the tree recursively: to evaluate an <code>Add</code> node, evaluate both children then add (post-order). Executes statements in order, handles control flow (if, loops, calls, return, exceptions), looks up and updates variables, calls functions, reports runtime errors (division by zero, type errors in dynamically typed languages, undefined variable).
Data structures	the AST; an environment = mapping from names to values, implemented as a hash table, with one table per scope linked to its enclosing scope (a chain of hash tables); a call stack of activation records (parameters, local variables, return address) for function calls; a heap for objects, closures, strings. A bytecode interpreter instead uses an array of instructions and a value stack.

INTERPRETER VS COMPILER (OFTEN ASKED TOGETHER)

A **compiler** translates the whole program to another language (e.g. machine code) *before* it runs; the lexer and parser are the same, but the last stage *generates code* instead of executing. An **interpreter** executes the program directly from its tree (or bytecode). Hybrid: compile to bytecode, then interpret it in a virtual machine (Java, Python), possibly with a JIT compiler. Interpreter: easier, portable, good error messages, slower. Compiler: faster execution, errors found before running, less flexible.

22.5 See it: a complete tiny interpreter (Haskell, runs)

For arithmetic like `2 + 3 * 4`. Look at what each stage produces.

Stage 1 — the lexer

```
data Token = TNum Int | TPlus | TMinus | TTimes | TLParen | TRParen
  deriving Show

-- LEXER: characters -> tokens
lexer :: String -> [Token]
lexer [] = []
lexer (c:cs)
  | isSpace c = lexer cs -- skip whitespace
  | isDigit c = TNum (read (c : takeWhile isDigit cs))
    : lexer (dropWhile isDigit cs)
  | c == '+' = TPlus : lexer cs
  | c == '-' = TMinus : lexer cs
  | c == '*' = TTimes : lexer cs
  | c == '(' = TLParen : lexer cs
  | c == ')' = TRParen : lexer cs
  | otherwise = error ("lexical error: unexpected " ++ [c])
```

```
ghci> lexer "2 + 3 * 4"
[TNum 2,TPlus,TNum 3,TTimes,TNum 4]
```

The syntax-tree type

```
data Expr = Num Int | Add Expr Expr | Sub Expr Expr | Mul Expr Expr
  deriving Show
```

Stage 2 — a recursive-descent parser: one function per grammar rule

```
-- PARSER: tokens -> syntax tree. Grammar (one function per rule):
-- expr  -> term { ("+" | "-") term }      lowest precedence, left-associative
-- term  -> factor { "*" factor }
-- factor -> NUMBER | "(" expr ")"         highest precedence

parse :: [Token] -> Expr
parse ts = case expr ts of
  (e, []) -> e
  (_, rest) -> error ("syntax error at " ++ show rest)

expr :: [Token] -> (Expr, [Token])
expr ts = let (t, rest) = term ts in loop t rest
  where
    loop acc (TPlus : r) = let (t, r') = term r in loop (Add acc t) r'
    loop acc (TMinus : r) = let (t, r') = term r in loop (Sub acc t) r'
    loop acc r = (acc, r)

term :: [Token] -> (Expr, [Token])
term ts = let (f, rest) = factor ts in loop f rest
  where
    loop acc (TTimes : r) = let (f, r') = factor r in loop (Mul acc f) r'
    loop acc r = (acc, r)

factor :: [Token] -> (Expr, [Token])
factor (TNum n : r) = (Num n, r)
factor (TLParen : r) = case expr r of
  (e, TRParen : r') -> (e, r')
  _ -> error "syntax error: expected )"
factor ts = error ("syntax error at " ++ show ts)
```

```
ghci> parse (lexer "2 + 3 * 4")
Add (Num 2) (Mul (Num 3) (Num 4))
ghci> parse (lexer "10 - 4 - 3")
Sub (Sub (Num 10) (Num 4)) (Num 3)
```

Look at the trees: `*` ended up **deeper** than `+` (higher precedence — done first), and `10 - 4 - 3` became `(10 - 4) - 3` (left associative). This is exactly what the grammar levels `expr → term → factor` produce.

Stage 3 — the executor walks the tree

```
-- EXECUTOR: walk the tree
eval :: Expr -> Int
eval (Num n)    = n
eval (Add a b)  = eval a + eval b
eval (Sub a b)  = eval a - eval b
eval (Mul a b)  = eval a * eval b

run :: String -> Int
run = eval . parse . lexer
```

```
ghci> run "2 + 3 * 4"
14
ghci> run "10 - 4 - 3"
3
ghci> run "(2 + 3) * 4"
20
```

22.6 Precedence and associativity (asked on both 2023 exams, 6 p)

MODEL ANSWER: “BRIEFLY DESCRIBE PRECEDENCE AND ASSOCIATIVITY. HOW ARE THEY RELATED IN THE CONTEXT OF PROGRAMMING LANGUAGES?”

Precedence decides which of two **different** operators binds tighter when there are no parentheses. `*` has higher precedence than `+`, so `2 + 3 * 4` means `2 + (3 * 4) = 14`, not `(2 + 3) * 4 = 20`.

Associativity decides how operators of the **same** precedence level are grouped. `-` is **left-associative**: `10 - 4 - 3 = (10 - 4) - 3 = 3`, not `10 - (4 - 3) = 9`. Exponentiation and assignment are usually **right-associative**: `2 ^ 3 ^ 2 = 2 ^ (3 ^ 2) = 512`; `a = b = 0 = a = (b = 0)`. Some operators are non-associative (`a < b < c` is an error in some languages).

Relation: both are rules for resolving the **ambiguity** of expressions written without parentheses, i.e. for deciding the shape of the syntax tree. Precedence is applied first (between levels); associativity is only needed when operators have **the same** precedence. In a grammar they are encoded structurally: one nonterminal per precedence level (`expr → expr + term | term`, `term → term * factor | factor`, `factor → num | ( expr )`) — lower levels are deeper in the tree, so they bind tighter; **left recursion** (`expr → expr + term`) gives left associativity, **right recursion** (`pow → base ^ pow`) gives right associativity. They matter because they change the result (`10-4-3`) and, with side effects, behaviour. Parentheses always override both.

MODEL ANSWER: “DESCRIBE AS MUCH AS THE SPACE PERMITS THE ROLE OF A PARSER IN A PROGRAM”

The parser is the second phase of an interpreter/compiler, after the lexer. Its input is the **sequence of tokens** produced by the lexer (e.g. ID(x) ASSIGN NUM(2) PLUS NUM(3) STAR NUM(4)). Its job is **syntax analysis**: it checks that the token sequence can be derived from the language's **context-free grammar**, and it builds a **syntax tree** (usually an abstract syntax tree) that represents the hierarchical structure of the program — which statements are inside which blocks, and which operands belong to which operators. When building the tree the parser applies the language's **precedence and associativity** rules, so $2+3*4$ becomes $+(2, *(3,4))$. If the tokens do not fit the grammar it reports a **syntax error** with its location and tries to recover to find further errors. The tree is the input to the later phases: semantic analysis (scopes, type checking, with a symbol table) and then execution (interpreter) or code generation (compiler). A common technique is **recursive descent**, where each grammar rule becomes a function and the recursion stack plays the role of the pushdown automaton's stack; other parsers are table-driven (LL, LR). Data structures used: the token list with a current position, a stack (explicit or the call stack), and the tree it outputs.

MODEL ANSWER: “AN INTERPRETER CONSISTS OF A SCANNER/LEXER, A PARSER AND AN EXECUTING PART. WHAT ARE THEIR TASKS AND WHAT DATA STRUCTURES DO THEY USE?” (8 P)

Lexer: reads the source text character by character and groups characters into lexemes, which it classifies as tokens (type, lexeme, value, line). Skips whitespace and comments, distinguishes keywords from identifiers, reports lexical errors. Works like a finite automaton defined by regular expressions. *Data structures:* input string/buffer with position pointers, keyword hash table, output list of tokens.

Parser: takes the token list and checks it against the context-free grammar of the language; builds a syntax tree (AST) that captures structure, precedence and associativity; reports syntax errors. Recursive descent or table-driven (LL/LR). *Data structures:* token list + index, a stack (call stack in recursive descent), the tree (and possibly a symbol table).

Executor: traverses the tree and carries out the computation: evaluates expressions recursively, executes statements, handles control flow and function calls, looks up/updates variables, reports runtime errors. *Data structures:* the AST, environments (hash tables mapping names to values, chained for nested scopes), a call stack of activation records (parameters, locals, return address), and a heap for dynamically allocated data.

23. Doing several things at once — and when things go wrong PL

23.1 Concurrency

Your phone plays music while you type a message and an app downloads in the background. Several things are happening at once. Sometimes truly at once (several processor cores), sometimes only apparently (one core switching between tasks very quickly). Either way, we call it **concurrency**.

Why bother? Because the world is concurrent (a program that models it naturally is too), because it keeps programs **responsive** (the interface keeps reacting while the heavy work happens in another thread), and because modern machines have many cores to use.

The problem: sharing

If concurrent tasks never talked to each other, there'd be no problem. The trouble starts when they **share data**. Picture two threads each doing `count = count + 1`, which is really three steps: read, add, write.

Thread A	Thread B	count
reads 5		5
	reads 5	5
writes 6		6
	writes 6	6

Two increments, but `count` went up by one. The result depends on timing — a **race condition**. It might work a thousand times and fail on the thousand-and-first. So languages need ways to **synchronise**: make sure only one task at a time is in a **critical section** (mutual exclusion), and let tasks wait for each other (cooperation).

The four tools

Tool	How it works	Trade-off
Semaphore	A counter with two atomic operations. wait : if the counter is above zero, decrease it and go on; otherwise block. signal : if someone is blocked, wake one; otherwise increase the counter.	Tiny and simple — but the programmer must get every wait/signal right. One mistake: deadlock or a race.
Monitor	A language construct that wraps shared data with its operations, and guarantees only one thread is inside at a time (like Java's <code>synchronized</code>).	Much safer; but can lock more than necessary, reducing parallelism.
Message passing	Tasks don't share memory at all; they send each other messages, like letters (synchronously or not).	Easy to understand, works across machines; costs speed and resources.
Transactional memory	Mark a block as a transaction: it either completes entirely, or is rolled back, as in a database.	Easy for the programmer; overhead, not available everywhere.

Two failure words to know: **deadlock** — A holds X and waits for Y, B holds Y and waits for X; both wait forever. **Starvation** — a task that never gets its turn.

23.2 Exceptions: when things go wrong

Things go wrong in three flavours: the truly unexpected (disk failure), the unexpected-but-possible (division by zero, overflow), and the expected (end of file). You *could* handle all of them by checking return values after every call — but code full of `if (error) ...` is hard to read. So modern languages use **try-catch**:

- the normal code is written cleanly inside `try`;
- when an error occurs, an exception is **raised** (thrown) and normal execution stops;
- if the current function has no matching `catch`, the exception **propagates** up the chain of callers until one does — so errors can be handled at whichever level knows what to do about them;
- `finally` runs cleanup code no matter what.

Designers have to decide: can exceptions carry information? how is the right handler chosen when there are several? where should handling happen? what happens if nobody catches it?

23.3 Events

An **event** is a cousin of an exception that isn't about errors: a button is clicked, a key is pressed, a timer fires, and a **handler** you registered earlier is called. Instead of the program deciding when to look for input, the input calls the program. That's how every graphical interface works.

24. Two other ways to think: functional and logic languages PL

24.1 The functional family

You've spent all of Part III inside Haskell, so you already know the functional way of thinking. Here it is in a sentence: a program is a set of **function definitions**, and a function's result depends **only on its arguments** — no hidden state, no side effects. The family tree: LISP (1958!) → Scheme, ML, Haskell, Erlang.

Here is the same function — “is `a` in the list?” — in three of them. Notice they all say the same thing: an empty list contains nothing; otherwise, check the first element, or look in the rest.

```
; Scheme
(DEFINE (member a lst)
  (COND ((NULL? lst) #F)
        ((EQ? a (CAR lst)) #T)
        (ELSE (member a (CDR lst)))))

(* ML *)
fun member a []      = false
  | member a (b::l) = a = b orelse member a l

-- Haskell
member a []      = False
member a (b:l) = a == b || member a l
```

Haskell goes further than most: it is **purely** functional and **lazy** — nothing is computed until it's needed, which is why an infinite list like `[n*n | n <- [0..]]` is perfectly fine.

WHY PEOPLE LOVE FUNCTIONAL LANGUAGES

Programs are easier to **prove correct** and to **transform** (a name always means the same value); functions are **first-class** (pass them around like data); the syntax and meaning are small and clean; and with no shared changing state, parallel execution is much safer. **Downsides:** on ordinary hardware they can be slower, and input/output and state are awkward. The ideas have spread everywhere: Java, JavaScript, Python, Ruby and C# all have lambdas and pass functions as parameters.

24.2 Logic programming: say what, not how

Every language so far makes you describe **how** to compute something. Logic programming flips that. You describe **what is true** — facts and rules — and then ask questions. A built-in search engine works out the answer.

Think about sorting. In Java you write the steps of an algorithm. In a logic language you could say: “*a sorted version of a list is a rearrangement of it in which every element is $\leq$ the next*” — and that description is the program.

Here's a small Prolog program. Facts first, then one rule: “X is a professor of Y if X teaches some course C and Y studies C”.

Now we ask three questions — what does charlie study? who are kirke's students? who teaches jack?

```
csc135  
charlie  
olivia  
collins
```

How did it find kirke's students? It looked for `teaches(kirke, C)` and found `C = csc135`. Then it looked for `studies(S, csc135)` and found charlie — and when asked for more, it **backtracked** and found olivia. Matching patterns with variables like this is called **unification**.

The catches: the blind search can be very slow; the order you write rules in still matters (a badly placed recursive rule can loop forever); answers are only “true” or “fail”; and “not” means “can't be proven”, which isn't the same as “false”. The **cut** (`!`) lets the programmer prune the search. Logic languages shine in databases, expert systems and AI.

Part V

Real exams solved, practice exams, cheat sheets

25. The FCS exam (5DV037), every question answered

FCS

Do it yourself first with a 4-hour timer, then compare.

Q1 (6 p) True/false — -0.5 for each wrong answer

Statement	Answer	Reason
Starts with a / ends with c / odd number of b's — regular	True	union of three regular languages (Ch. 3 regex)
L CF non-regular, L' regular $\Rightarrow L \cup L'$ not regular	False	$L' = \Sigma^*$
L CF non-regular, L' regular $\Rightarrow L \cap L'$ not regular	False	$L' = \emptyset$
$P = NP \Rightarrow$ poly-time algorithm for SAT	True	$SAT \in NP = P$
It can be correct to say a TM M is decidable	False	languages/problems are decidable, not machines
Set of encoded TMs is RE	True	even recursive: a TM checks the encoding

Q2 (5 p) closure: model answer in Ch. 5.1. **Q3** (8 p) hierarchy table: Ch. 12.4.

Q4 (8 p): as printed, the language is context-free, so it can't be proved non-CF — see Ch. 11 for what to write, and the proof technique for languages that really are non-CF ($a^n b^n c^n$).

Q5 (8 p) CNF: **Yes**, it is in CNF. CYK: `abca` $\in L(G)$. Full table in Ch. 10, Part B.

Q6 (6 p) M_{gf} : Ch. 12.1. **Q7** (6 p) NTM vs DTM: Ch. 12.2.

Q8 ☆ ◇-EQUATION: Ch. 14.3. **Q9** ☆ infinite union: **not** necessarily regular, Ch. 5.4. **Q10** ☆ NP closure: Ch. 14.4.

25.1 Practice exam (new questions)

- (6 p) True/false: (a) Every regular language is context-free. (b) The complement of a CFL is CF. (c) If L is RE and $\bar{L}$ is RE then L is recursive. (d) $\{a^n b^n\} \cap a^* b^*$ is regular. (e) NFAs accept more languages than DFAs. (f) If $A \leq_p B$ and $B \in P$ then $A \in P$.
- (6 p) Give a DFA for strings over $\{a,b\}$ that contain ab and have even length.
- (6 p) Convert the NFA $p_0 \xrightarrow{a,b} p_0$, $p_0 \xrightarrow{b} p_1$, $p_1 \xrightarrow{a} p_2$ (final p_2) to a DFA.
- (8 p) Prove $\{a^n b^m \mid n > m\}$ is not regular.
- (8 p) Convert $S \rightarrow aSb \mid SS \mid \lambda$ to CNF and run CYK on `aabb`.
- (8 p) Prove $\{a^n b^n c^n\}$ is not context-free.
- (6 p) Show that the problem “does TM M ever write the symbol 1 on input w ?” is undecidable.

SOLUTIONS

1. (a) T (b) F (c) T (d) F — it equals $\{a^n b^n\}$ (e) F (f) T.
2. Product of “seen ab” (3 states: none / last was a / seen) and parity (2 states): 6 states, final = (seen, even).
3. Language: ends in ba . DFA states $\{p_0\}, \{p_0, p_1\}, \{p_0, p_2\}$: $\{p_0\} \xrightarrow{a} \{p_0\}, \{p_0\} \xrightarrow{b} \{p_0, p_1\}, \{p_0, p_1\} \xrightarrow{a} \{p_0, p_2\}, \{p_0, p_1\} \xrightarrow{b} \{p_0, p_1\}, \{p_0, p_2\} \xrightarrow{a} \{p_0\}, \{p_0, p_2\} \xrightarrow{b} \{p_0, p_1\}$; final $\{p_0, p_2\}$.
4. $w = a^{n+1}b^n; y = a^j; i = 0$ gives $a^{n+1-j}b^n$ with $n + 1 - j \leq n$: not in L.
5. Nullable S. Remove λ : $S \rightarrow aSb \mid ab \mid SS$ (and $S \rightarrow S$, dropped). CNF: $S \rightarrow AX \mid AB \mid SS, X \rightarrow SB, A \rightarrow a, B \rightarrow b$. CYK on aabb: row1 A,A,B,B; row2 $\emptyset, S, \emptyset$; row3 $\emptyset$ (a|abb: $A \cdot \emptyset$; aa|bb: $\emptyset$), X (abb = ab|b: $S \cdot B \rightarrow X$); row4: a|abb: $A \cdot X \rightarrow S$. $S \in \text{top} \Rightarrow$ accepted.
6. Ch. 11 Example 1.
7. Reduce halting: given M, w, build M' that simulates M using a tape alphabet without 1 (rename 1 to a new symbol), and when M would halt, writes 1. M' writes 1 on w $\Leftrightarrow$ M halts on w.

26. The PL exams (5DV086), every question answered

PL

26.1 Tentamen 2023-03-16

#	Question (translated)	Answer
1 (10)	Which strings can G generate?	None of them. $L(G) = \{a^{k+1}b^k\} \cup \{cb\}$. Ch. 7.3
2 (10)	CFG for “equal a and b, every prefix $\geq$ as many a as b”	$S \rightarrow aSbS \mid \varepsilon$. Ch. 7.3
3 (6)	Precedence and associativity; how related?	Model answer Ch. 8
4 (8)	Parts of an interpreter, tasks, data structures	Model answer Ch. 22
5 (6)	Pass-by-reference vs pass-by-value, pros/cons	Model answer Ch. 20.3
6 (10)	Write <code>compress</code>	Ch. 16 (both versions)
7 (10)	Write <code>myRev</code> , $O(n)$, no library functions	Ch. 16

26.2 Tentamen 2023-05-05

#	Question	Answer
1 (10)	CFG for $\{a^n b^m \mid n \neq 2m\}$	Ch. 7.3 (both readings — say which one you answer)
2–4, 7	same as above	
5 (6)	Why is <code>a = b + 4</code> assignment vs naming a value?	Model answer Ch. 19.3
6 (10)	What do <code>foo</code> , <code>bar</code> do; evaluate them	<code>"cccc"</code> and 8. Ch. 17.3

26.3 Your last exam (what you told me)

Topic	Where
Convert to Chomsky normal form	Ch. 10 Part A
CYK: show table, explain	Ch. 10 Part B
Encapsulation / information hiding	Ch. 21
Role of the parser	Ch. 22
Write <code>foldl</code> ; list to int	Ch. 17.3, Ch. 16
Call by value vs reference	Ch. 20.3
decompress	Ch. 16

Pattern: the PL exam = **grammars (CFG design / CNF / CYK) + 3 theory questions + 2–3 Haskell functions.**

26.4 Practice exam (60 p, 36 to pass) — 4 hours, no aids

1. (10 p) Construct a CFG for $\{a^n b^m c^k \mid k = n + m\}$. Then show a leftmost derivation of $abcc$.
2. (10 p) Convert $S \rightarrow aSb \mid bA \mid \lambda$, $A \rightarrow aA \mid a$ to Chomsky normal form.
3. (6 p) Explain static vs dynamic scoping with an example where they give different results.
4. (8 p) What is polymorphism? Describe three kinds with examples.
5. (6 p) What is the difference between a compiler and an interpreter? What does the lexer produce?
6. (10 p) Write `countRuns :: String -> Int` that counts the number of maximal runs of equal characters (`countRuns "aaabbca" = 4`), and `myFoldr`.
7. (10 p) Write `merge :: [Int] -> [Int] -> [Int]` merging two sorted lists, and use it to write `merge sort`.

SOLUTIONS

1. $S \rightarrow aSc \mid B$, $B \rightarrow bBc \mid \lambda$ (R5). $S \Rightarrow aSc \Rightarrow aBc \Rightarrow abBcc \Rightarrow abcc$.
2. Nullable: S. Remove λ : $S \rightarrow aSb \mid ab \mid bA$, $A \rightarrow aA \mid a$. No units. CNF: $S \rightarrow UX \mid UT \mid TA$, $X \rightarrow ST$, $A \rightarrow UA \mid a$, $U \rightarrow a$, $T \rightarrow b$.
3. Ch. 19.4 (the Python/bash example). 4. Ch. 21.3. 5. Ch. 22.
6. `countRuns = length . compress` (using `compress` from Ch. 16), or directly: `countRuns [] = 0`;
`countRuns (c:cs) = 1 + countRuns (dropWhile (== c) cs)`. `myFoldr` = `foldr` from Ch. 17.
7. Ch. 16 (`merge`, `mergeSort`).

Appendix A. Cheat sheets

AUTOMATA & REGULAR

- DFA 5-tuple; δ total. NFA: $\delta \rightarrow$ sets, λ -moves. NFA $\rightarrow$ DFA: sets of states + λ -closure; final if contains a final.
- Regex: star > concat > union (+).
- Closed: $\cup$ $\cap$ complement concat star reverse difference. Complement needs a complete DFA.
- Pumping: $w = xyz$, $|xy| \leq n$, $|y| \geq 1$. Pick w from n ; all splits; pump 0 or 2.
- Minimize: mark final/non-final pairs; repeat; merge unmarked.

CONTEXT-FREE

- aSb matches pairs. $\neq = < \cup >$. "or" = $S \rightarrow S_1 \mid S_2$.
- Balanced: $S \rightarrow aSbS \mid \lambda$. Equal count: $S \rightarrow aSbS \mid bSaS \mid \lambda$.
- CNF: $\lambda \rightarrow$ units $\rightarrow$ useless $\rightarrow$ terminals $\rightarrow$ split.
- CYK: V_{ij} from $A \rightarrow a$; V_{ij} over all cuts k with $A \rightarrow BC$; $S \in V_{1n} = \text{member}$.
- CF pumping: $xuyvz$, $|uyv| \leq n$, $|uv| \geq 1$, window anywhere.
- CFL closed: $\cup$, concat, star, $\cap$ regular. Not: $\cap$, complement.

COMPUTABILITY

- TM 7-tuple; halts when δ undefined.
- Regular $\subset$ DCFL $\subset$ CFL $\subset$ (CSL) $\subset$ Recursive $\subset$ RE.
- L and $\bar{L}$ RE $\Rightarrow$ L recursive. Halting: RE, not recursive; its complement not RE.
- Reduce known-undecidable $\rightarrow$ new problem.
- NP: poly verifier / NTM. NP-complete: in NP + reduction from known NPC.

HASKELL

- Signature, base case, recursive case, trace.
- Accumulator: `go xs acc`; `go [] acc = acc`.
- `foldl f acc (x:xs) = foldl f (f acc x) xs`
- `foldr f acc (x:xs) = f x (foldr f acc xs)`
- $x : xs$ is $O(1)$; $xs ++ ys$ is $O(|xs|)$.
- `listToInt`: `acc*10 + d`. `decompress`: `replicate n c ++ rest`.

PL CONCEPTS

- Encapsulation = grouping in one unit. Information hiding = internals inaccessible.
- Polymorphism: subtype, parametric, ad hoc.
- Lexer $\rightarrow$ tokens; parser $\rightarrow$ AST (+ precedence); executor $\rightarrow$ environments, call stack, heap.
- Value = copy; reference = alias; result; value-result; name.
- Static scope = text; dynamic = call chain.
- Semaphore wait/signal; monitor; messages; transactional memory.

Appendix B. Swedish–English glossary (PL exam is in Swedish)

Svenska	English	Svenska	English
uppgift	problem / question	poäng	points
konstruera	construct	kontextfri grammatik	context-free grammar
generera	generate	sträng / språk	string / language
icketerminal / terminal	non-terminal / terminal	startsymbol	start symbol
regel	rule / production	tomma strängen	the empty string
prefix	prefix	minst lika många	at least as many
hälften	half	dubbelt så många	twice as many
beskriv kortfattat	describe briefly	förklara	explain
precedens / associativitet	precedence / associativity	tolk / interpretator	interpreter
kompilator	compiler	datastruktur	data structure
tilldelning	assignment	binda ett värde till ett namn	bind a value to a name
sidoeffekt	side effect	skillnad	difference
för- och nackdelar	advantages and disadvantages	värdeanrop / referensanrop	call by value / by reference
inkapsling	encapsulation	informationsdöljande	information hiding
arv	inheritance	dynamisk bindning	dynamic binding
skriv en funktion	write a function	hjälpfunktion	helper function
inbyggda funktioner	built-in functions	tidskomplexitet, linjär	time complexity, linear
vänder på listor	reverses lists	svansrekursiv	tail-recursive
upprepningar	repetitions	returvärde	return value
jämnlöpande / samtidighet	concurrent / concurrency	avbrott / undantag	interrupt / exception
räckvidd	scope	livslängd	lifetime